Robot Perception and Learning

Value Function Approximation, Policy Gradient

Tsung-Wei Ke

Fall 2026



Disclaimer

- This lecture heavily borrows contents from:
  - RL-Trust Region Policy Optimization (TRPO) Explained by Jonathan Hui (link)
  - Reinforcement Learning by Ping-Chun Hsieh at National Yang Ming Chiao Tung University.

Content

- Introduction to Deep RL
- Deep Q-Learning
- Policy Gradient
  - REINFORCE
  - Actor-Critic Methods
  - PPO

Content

- Introduction to Deep RL
- Deep Q-Learning
- Policy Gradient
  - REINFORCE
  - Actor-Critic Methods
  - PPO

Tabular Value Function Learning

- TD state-value learning:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- Q-learning:

$$\mathrm{Q}(S_t, A_t) \leftarrow \mathrm{Q}(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a \mathrm{Q}(S_{t+1}, a\,) - \mathrm{Q}(S_t, A_t)]$$

- We used a tabular setup: the value is retrieved from a table with a key of state/state-action pair

- What are the limitations?

Tabular Value Function Learning

- TD state-value learning:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- Q-learning:

$$\mathrm{Q}(S_t, A_t) \leftarrow \mathrm{Q}(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a \mathrm{Q}(S_{t+1}, a\,) - \mathrm{Q}(S_t, A_t)]$$

- We used a tabular setup: the value is retrieved from a table with a key of state/state-action pair

- What are the limitations?
  - Discrete state/action space

Tabular Value Function Learning

- TD state-value learning:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- Q-learning:

$$\mathrm{Q}(S_t, A_t) \leftarrow \mathrm{Q}(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a \mathrm{Q}(S_{t+1}, a\,) - \mathrm{Q}(S_t, A_t)]$$

- We used a tabular setup: the value is retrieved from a table with a key of state/state-action pair

- What are the limitations?
  - Discrete state/action space
  - Curse of dimensionality: too many states / state-action pairs stored in the memory

$$\mathbf{s} = 0 : V(\mathbf{s}) = 0.2$$
$$\mathbf{s} = 1 : V(\mathbf{s}) = 0.3$$
$$\mathbf{s} = 2 : V(\mathbf{s}) = 0.5$$



$$|\mathcal{S}| = (255^3)^{200 \times 200}$$

(more than atoms in the universe)

Tabular Value Function Learning

- TD state-value learning:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

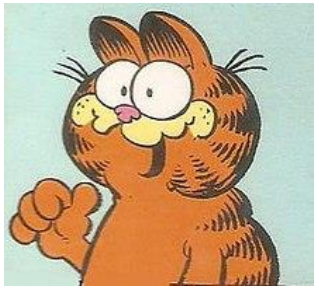
- Q-learning:

$$\mathrm{Q}(S_t, A_t) \leftarrow \mathrm{Q}(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a \mathrm{Q}(S_{t+1}, a) - \mathrm{Q}(S_t, A_t)]$$

- We used a tabular setup: the value is retrieved from a table with a key of state/state-action pair

- What are the limitations?
  - Discrete state/action space
  - Curse of dimensionality: too many states / state-action pairs stored in the memory
  - Closed world (can't generalize to unseen state/action)





Tabular Value Function Learning

- TD state-value learning:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- Q-learning:

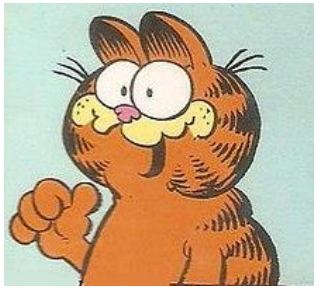
What are the labels Fit an approximator

$$\mathrm{Q}(S_t, A_t) \leftarrow \mathrm{Q}(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a \mathrm{Q}(S_{t+1}, a) - \mathrm{Q}(S_t, A_t)]$$

- We used a tabular setup: the value is retrieved from a table with a key of state/state-action pair

- What are the limitations?
  - Discrete state/action space
  - Curse of dimensionality: too many states / state-action pairs stored in the memory
  - Closed world (can't generalize to unseen state/action)





Content

- Introduction to Deep RL
- Deep Q-Learning
- Policy Gradient
  - REINFORCE
  - Actor-Critic Methods
  - PPO

Value Function Approximation

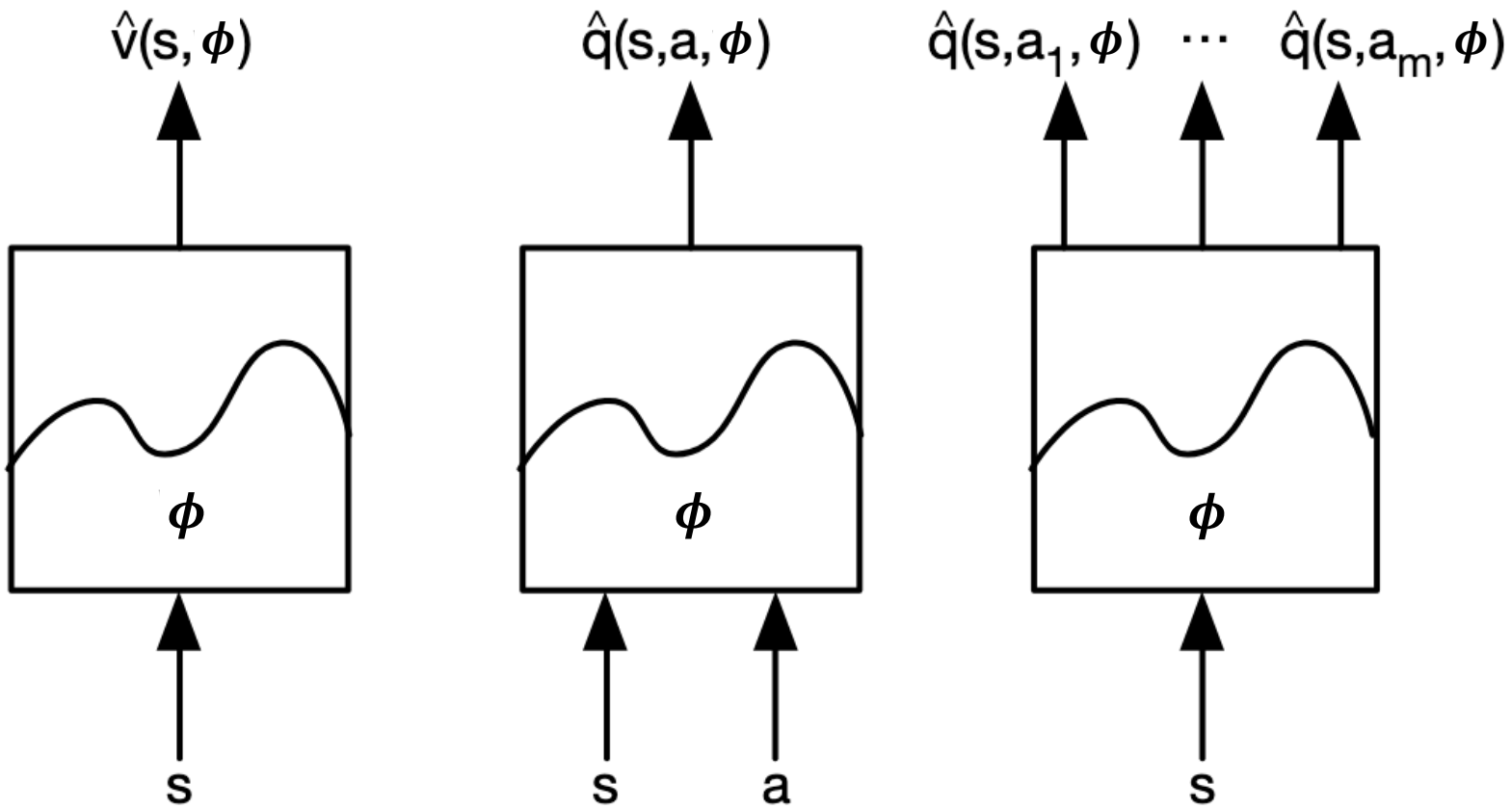


Image credit D. Silver

Value Function Approximation by Stochastic Gradient Descent

- Goal: find parameter vector $\boldsymbol{\phi}$ minimizing mean-squared error between approximate value function $\hat{V}(S, \boldsymbol{\phi})$ and true value function $V^{\pi}(S)$

$$J(\boldsymbol{\phi}) = \mathbb{E}_{\pi}\left[(V^{\pi}(S) - \hat{V}(S, \boldsymbol{\phi}))^2\right]$$

- Let $\mu(S)$ denote how much time we spend in each state S under policy π, then

$$J(\boldsymbol{\phi}) = \sum_{s \in \mathcal{S}}^{|\mathcal{S}|} \mu(S)\left[V^{\pi}(S) - \hat{V}(S, \boldsymbol{\phi})\right]^2 \qquad \text{s.t} \qquad \sum_{s \in \mathcal{S}} \mu(S) = 1$$

- In contrast to

$$J(\boldsymbol{\phi}) = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}}^{|\mathcal{S}|}\left[V^{\pi}(S) - \hat{V}(S, \boldsymbol{\phi})\right]^2$$

Slide credit K. Fragkiadaki

Value Function Approximation by Stochastic Gradient Descent

- Goal: find parameter vector $\boldsymbol{\phi}$ minimising mean-squared error between approximate value function $\hat{V}(S, \boldsymbol{\phi})$ and true value function $V^{\pi}(S)$

$$J(\boldsymbol{\phi}) = \mathbb{E}_{\pi}\left[(V^{\pi}(S) - \hat{V}(S, \boldsymbol{\phi}))^2\right]$$

- Let $\mu(S)$ denote how much time we spend in each state S under policy π, then

$$J(\boldsymbol{\phi}) = \sum_{s \in \mathcal{S}}^{|\mathcal{S}|} \mu(S)\left[V^{\pi}(S) - \hat{V}(S, \boldsymbol{\phi})\right]^2 \qquad \text{s.t} \qquad \sum_{s \in \mathcal{S}} \mu(S) = 1$$

- In contrast to

We care more about the frequently visited states, even though the total number of states is huge

$$J(\boldsymbol{\phi}) = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}}^{|\mathcal{S}|} \left[V^{\pi}(S) - \hat{V}(S, \boldsymbol{\phi})\right]^2$$

On-policy State Distribution

- Let $h(S)$ be the initial state distribution, i.e, the probability that an episode starts at state S
- Un-normalized on-policy state probability satisfies the following recursions:

$$\eta(S) = h(S) + \sum_{S'} \eta(S') \sum_{a} \pi(a|S') p(S|S', a)$$

$$\mu(S) = \frac{\eta(S)}{\sum_{S'} \eta(S')}$$

Slide credit K. Fragkiadaki

What are the Targets of Value Function Approximation?

- We usually have no access to the true value function $V^{\pi}(S)$ (otherwise, the problem is solved). Let $\mathbf{y}(s)$ be the target of the value function approximator

$$J(\boldsymbol{\phi}) = \mathbb{E}_{\pi}\left[(y(S) - \hat{V}(S, \boldsymbol{\phi}))^2\right]$$

- What could be $\mathbf{y}(s)$?
  - Monte-Carlo Method?
  - Temporal Difference Method?

Slide credit D. Silver

Recap: Monte-Carlo and Temporal Difference Method

- Monte Carlo (MC) methods: must wait until the end of the episode to learn value functions (only when the return is known)

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

- Temporal-Difference (TD) methods: combine Monte Carlo methods with Dynamic Programming methods that wait only until the next time step and bootstrap value functions from existing estimates

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

Slide credit D. Silver

Recap: Monte-Carlo and Temporal Difference Method

- Monte Carlo (MC) methods: must wait until the end of the episode to learn value functions (only when the return is known)

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

- Temporal-Difference (TD) methods: combine Monte Carlo methods with Dynamic Programming methods that wait only until the next time step and bootstrap value functions from existing estimates

This could be the target!

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

Slide credit D. Silver

Monte-Carlo with Value Function Approximation

Return G_t is unbiased, noisy sample of true value $V_\pi(S_t)$

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\boldsymbol{\phi} \in \mathbb{R}^d$ arbitrarily (e.g., $\boldsymbol{\phi} = \mathbf{0}$)

Loop forever (for each episode):
    Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π
    Loop for each step of episode, $t = 0, 1, \ldots, T-1$:
        $\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} + \alpha\big[G_t - \hat{v}(S_t, \boldsymbol{\phi})\big]\nabla\hat{v}(S_t, \boldsymbol{\phi})$

Temporal Difference with Value Function Approximation

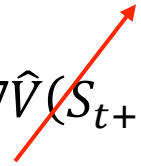
- Objective:

$$J(\boldsymbol{\phi}) = \mathbb{E}_{\pi}\left[(R_{t+1} + \gamma \hat{V}(S_{t+1}, \boldsymbol{\phi}) - \hat{V}(S_t, \boldsymbol{\phi}))^2\right]$$

- What are the gradients?

$$\nabla J(\boldsymbol{\phi}) = \mathbb{E}_{\pi}\left[(R_{t+1} + \gamma \hat{V}(S_{t+1}, \boldsymbol{\phi}) - \hat{V}(S_t, \boldsymbol{\phi}))^2\right]$$

$$= \mathbb{E}_{\pi}\left[\left(R_{t+1} + \gamma \hat{V}(S_{t+1}, \boldsymbol{\phi}) - \hat{V}(S_t, \boldsymbol{\phi})\right)\left(\gamma \nabla \hat{V}(S_{t+1}, \boldsymbol{\phi}) - \nabla \hat{V}(S_t, \boldsymbol{\phi})\right)\right]$$



Ignore the dependence of the target on $\boldsymbol{\phi}$!
This is called semi-gradient method

Temporal Difference with Value Function Approximation

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal},\cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\boldsymbol{\phi} \in \mathbb{R}^d$ arbitrarily (e.g., $\boldsymbol{\phi} = \mathbf{0}$)

Loop for each episode:
    Initialize S
    Loop for each step of episode:
        Choose $A \sim \pi(\cdot|S)$
        Take action A, observe R, S'
        $\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} + \alpha\big[R + \gamma\hat{v}(S', \boldsymbol{\phi}) - \hat{v}(S, \boldsymbol{\phi})\big]\nabla\hat{v}(S, \boldsymbol{\phi})$
        $S \leftarrow S'$
    until S is terminal

How About Action-Value Function Approximation? Same Story!

- Goal: find parameter vector $\boldsymbol{\phi}$ minimizing mean-squared error between approximate action-value function $\hat{Q}(S, A, \boldsymbol{\phi})$ and true value function $Q_\pi(S, A)$

$$J(\boldsymbol{\phi}) = \mathbb{E}_\pi\left[(Q_\pi(S, A) - \hat{Q}(S, A, \boldsymbol{\phi}))^2\right]$$

- Use Stochastic Gradient Descent to optimize $\boldsymbol{\phi}$:

$$-\frac{1}{2}\nabla_{\boldsymbol{\phi}} J(\boldsymbol{\phi}) = \left(Q_\pi(S, A) - \hat{Q}(S, A, \boldsymbol{\phi})\right)\nabla_{\boldsymbol{\phi}}\hat{Q}(S, A, \boldsymbol{\phi})$$

$$\Delta\boldsymbol{\phi} = \alpha\left(Q_\pi(S, A) - \hat{Q}(S, A, \boldsymbol{\phi})\right)\nabla_{\boldsymbol{\phi}}\hat{Q}(S, A, \boldsymbol{\phi})$$

How About Action-Value Function Approximation? Same Story!

- Goal: find parameter vector $\boldsymbol{\phi}$ minimizing mean-squared error between approximate action-value function $\hat{Q}(S, A, \boldsymbol{\phi})$ and true value function $Q_\pi(S, A)$

$$J(\boldsymbol{\phi}) = \mathbb{E}_\pi\left[\left(Q_\pi(S, A) - \hat{Q}(S, A, \boldsymbol{\phi})\right)^2\right]$$

- For MC:

$$J(\boldsymbol{\phi}) = \mathbb{E}_\pi\left[\left(G_t - \hat{Q}(S_t, A_t, \boldsymbol{\phi})\right)^2\right]$$

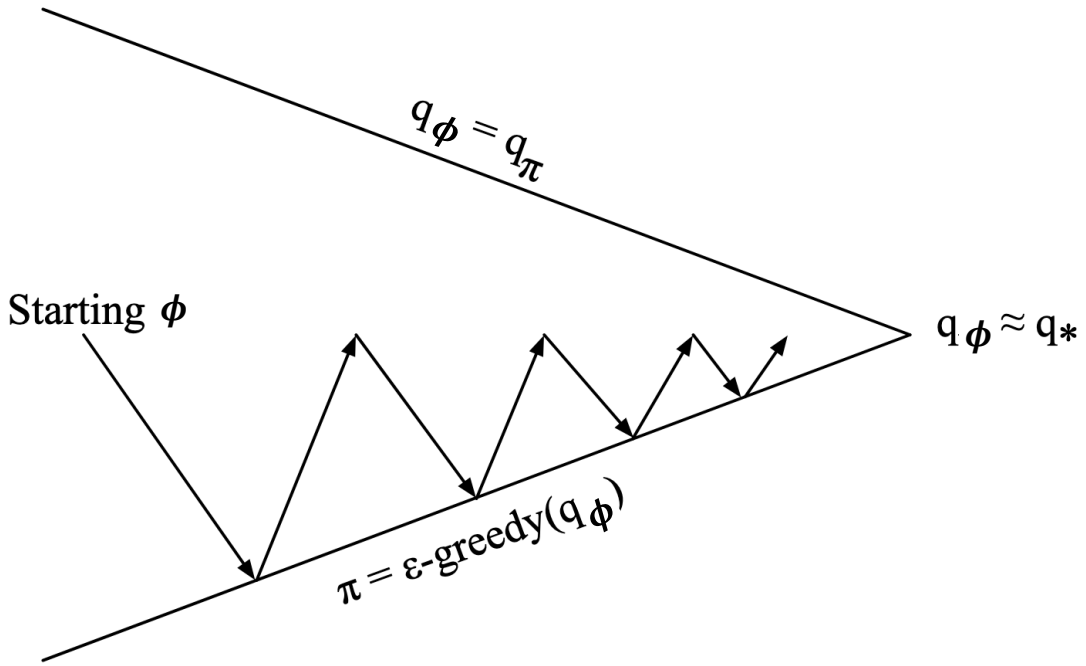
- For TD(0):

$$J(\boldsymbol{\phi}) = \mathbb{E}_\pi\left[\left(R_{t+1} + \gamma\hat{Q}(S_{t+1}, A_{t+1}, \boldsymbol{\phi}) - \hat{Q}(S_t, A_t, \boldsymbol{\phi})\right)^2\right]$$

- For Q-learning:

$$J(\boldsymbol{\phi}) = \mathbb{E}_\pi\left[\left(R_{t+1} + \gamma \max_a \hat{Q}(S_{t+1}, a, \boldsymbol{\phi}) - \hat{Q}(S_t, A_t, \boldsymbol{\phi})\right)^2\right]$$

Control with Action-Value Function Approximation



Policy evaluation Approximate policy evaluation, $\hat{q}(\cdot, \cdot, \boldsymbol{\phi}) \approx q_\pi$

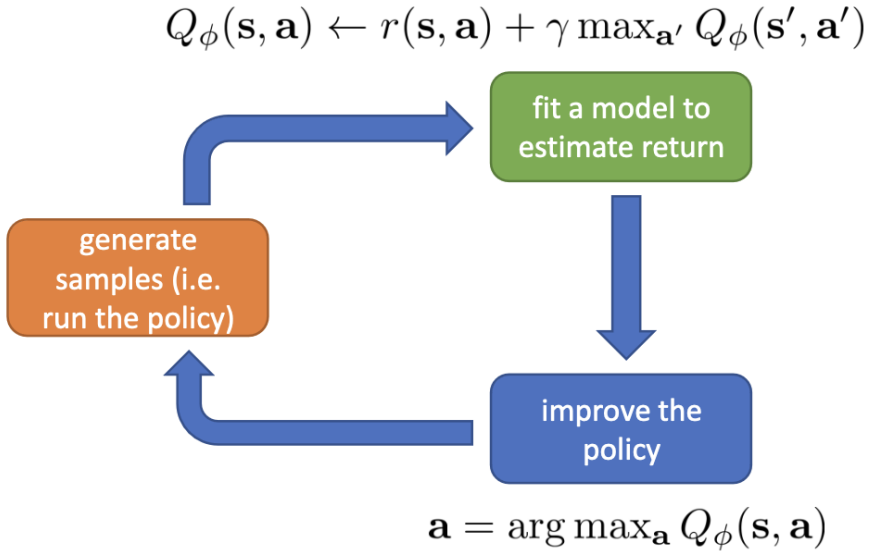
Policy improvement ϵ-greedy policy improvement

Slide credit D. Silver

So Far, We Discussed an Online Setup of Q-Learning

online Q iteration algorithm:

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$



Slide adapted from S. Levine

Problems in Online Q-Learning

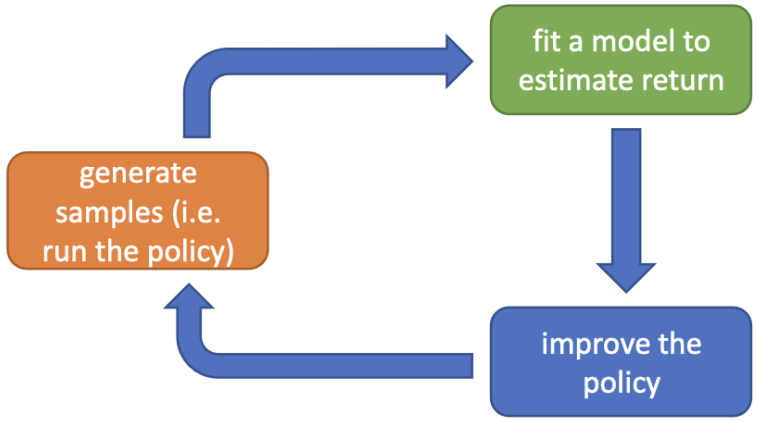
online Q iteration algorithm:

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$

1. Sequential samples are highly correlated (little information gain)



$Q_\phi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}', \mathbf{a}')$



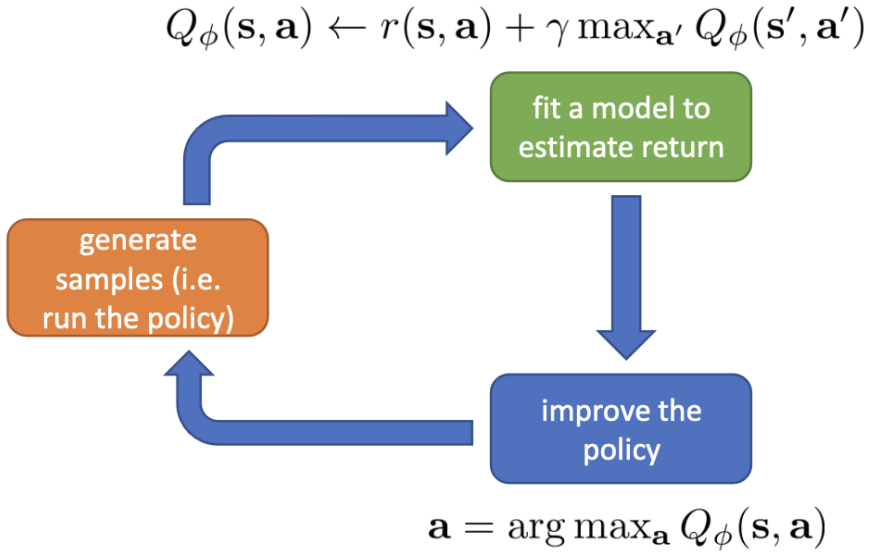
$\mathbf{a} = \arg\max_{\mathbf{a}} Q_\phi(\mathbf{s}, \mathbf{a})$

Problems in Online Q-Learning

online Q iteration algorithm:

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$

1. Sequential samples are highly correlated (little information gain)
2. Target value changes whenever ϕ is updated



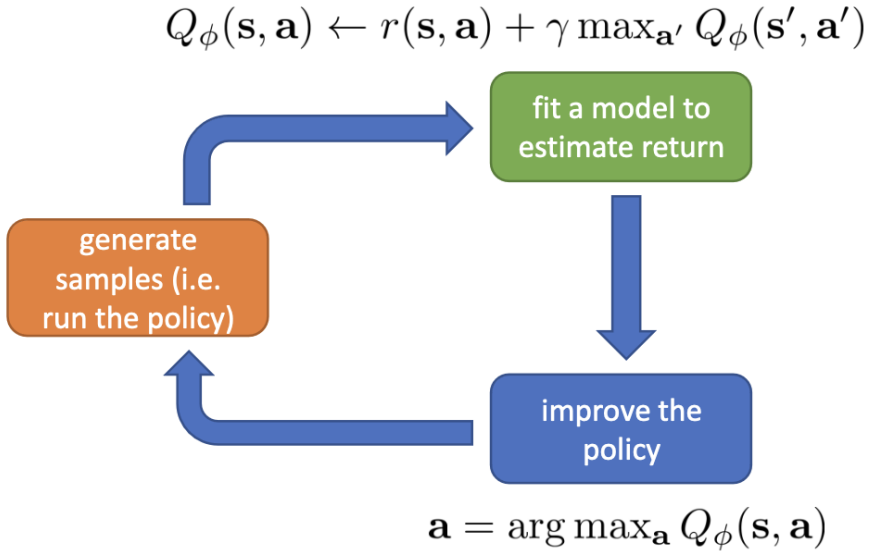
Problems in Online Q-Learning

online Q iteration algorithm:

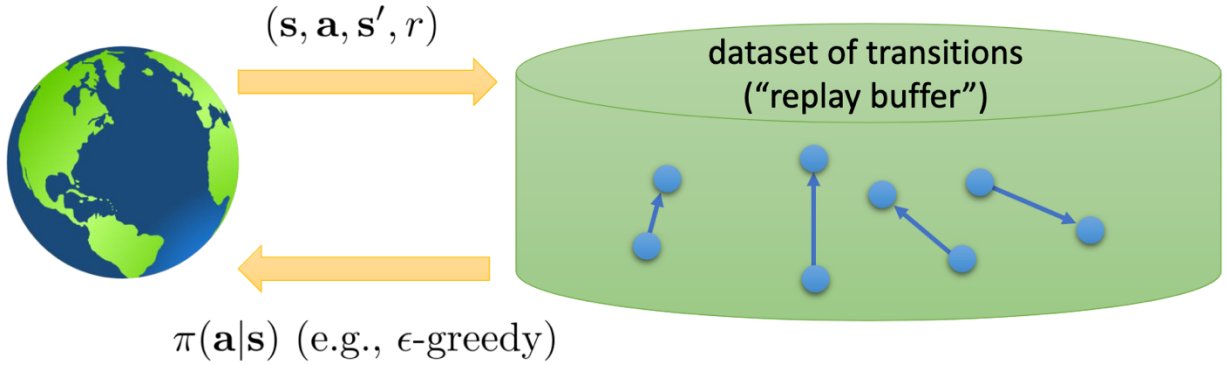
1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$

$$\phi \leftarrow \phi + \alpha \frac{dQ_\phi(s_i, a_i)}{d\phi} \left(Q_\phi(s_i, a_i) - r(s_i, a_i) - \gamma \max_{a'} Q_\phi(s'_i, a'_i)\right)$$

1. Sequential samples are highly correlated (little information gain)
2. Target value changes whenever ϕ is updated
3. Gradients don't pass through $Q_\phi(s'_i, a'_i)$ (but it becomes numerically unstable when propagating gradients through $Q_\phi(s'_i, a'_i)$)



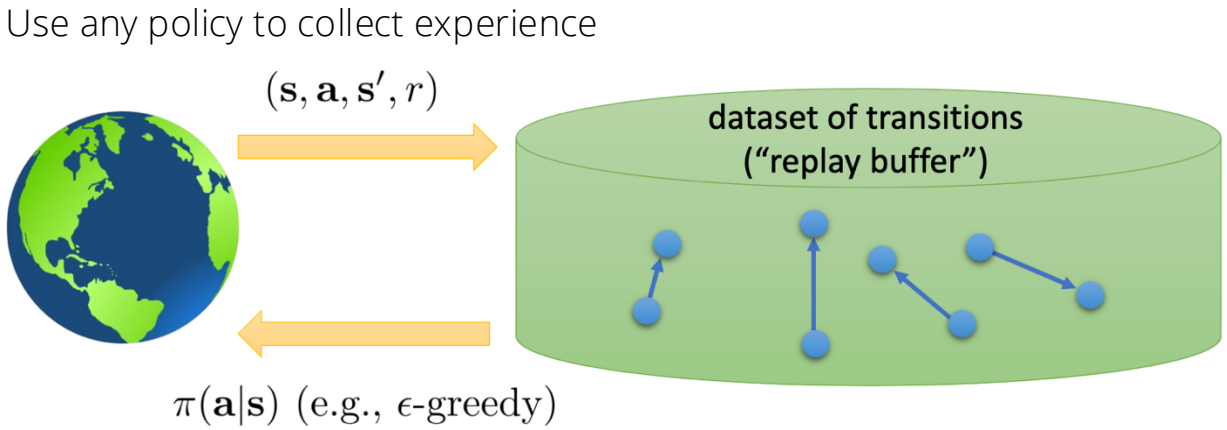
Solution 1: Use Replay Buffer to Decorrelate Samples



- Collect transitions data, using a random policy for exploration (find unseen states and transitions), and a deterministic policy for exploitation (obtain most promising states)

Slide adapted from S. Levine

Solution 1: Use Replay Buffer to Decorrelate Samples

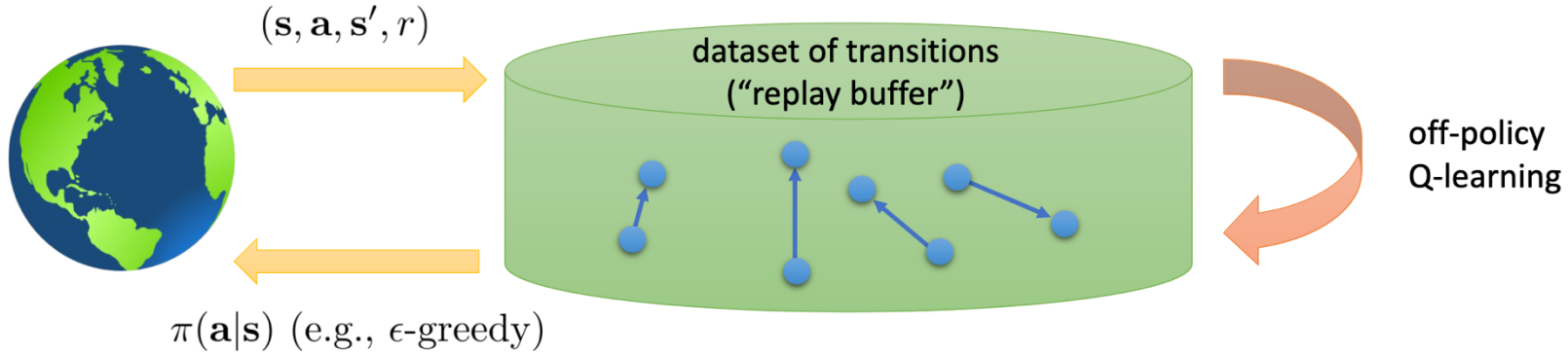


- Collect transitions data, using a random policy for exploration (find unseen states and transitions), and a deterministic policy for exploitation (obtain most promising states)

- Sample a batch of $\mathcal{B} = \{(s, a, s', r)\}$ from every collected (s, a, s', r), temporally neighboring (s, a, r) may not be sampled.

Slide adapted from S. Levine

Solution 1: Use Replay Buffer to Decorrelate Samples



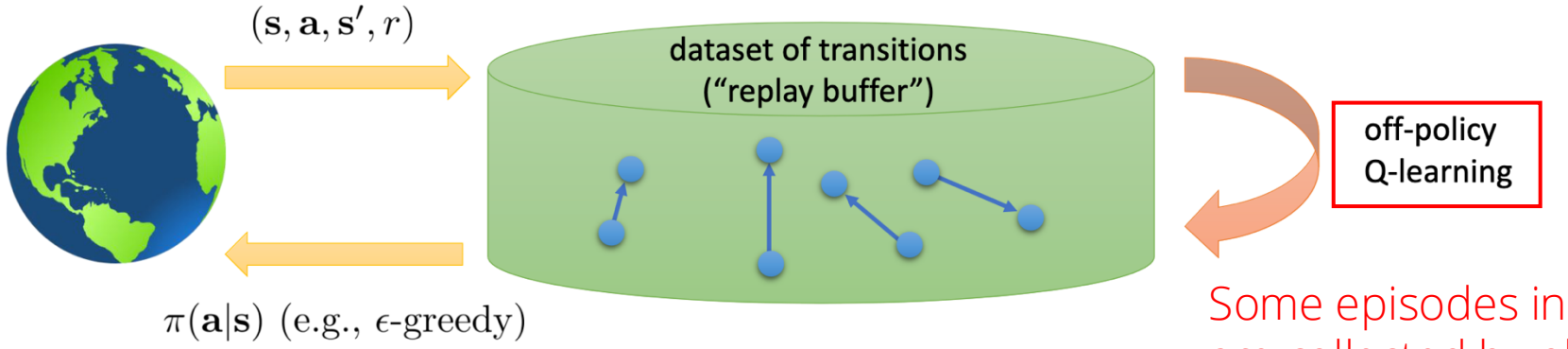
full Q-learning with replay buffer:

1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy, add it to $\mathcal{B}$
2. sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$ from $\mathcal{B}$
3. $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)])$

$K\times$

Slide adapted from S. Levine

Solution 1: Use Replay Buffer to Decorrelate Samples



Some episodes in the replay buffer are collected by older policies

full Q-learning with replay buffer:

1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy, add it to $\mathcal{B}$
2. sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$ from $\mathcal{B}$
3. $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)])$

$K\times$

Slide adapted from S. Levine

Fully Fitted Q-Iteration vs. Online Q-Learning

online Q iteration algorithm:

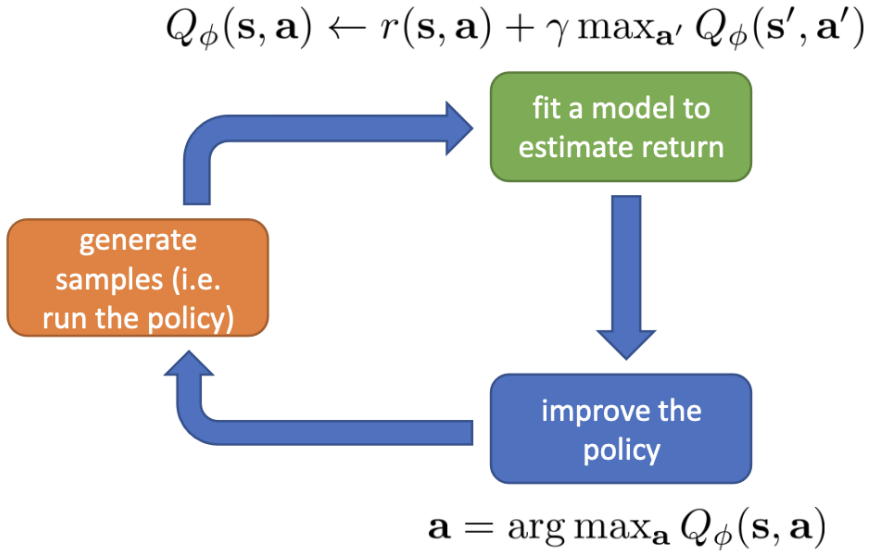
1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$

full fitted Q-iteration algorithm:

1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy

$K\times$

2. set $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. set $\phi \leftarrow \arg\min_\phi \frac{1}{2} \sum_i \|Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\|^2$



Fully Fitted Q-Iteration vs. Online Q-Learning

online Q iteration algorithm:

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$

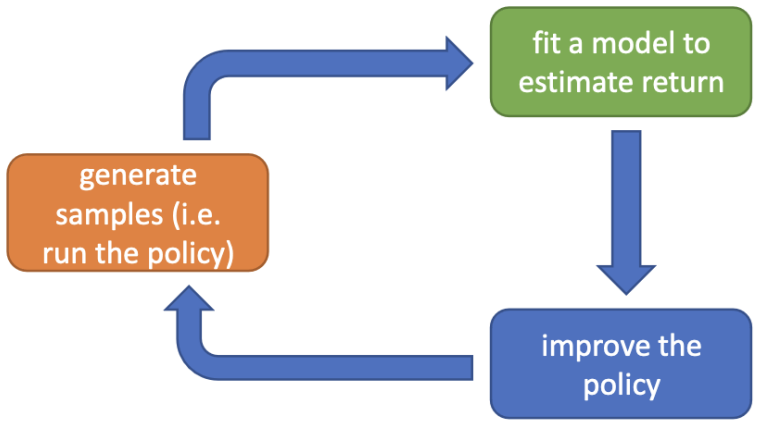
full fitted Q-iteration algorithm:

1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy

$K\times$

2. set $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. set $\phi \leftarrow \arg\min_\phi \frac{1}{2} \sum_i \|Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\|^2$

$Q_\phi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}', \mathbf{a}')$



$\mathbf{a} = \arg\max_{\mathbf{a}} Q_\phi(\mathbf{s}, \mathbf{a})$

We still have these problems:

2. Target value changes whenever ϕ is updated
3. Gradients don't pass through $Q_\phi(s'_i, a'_i)$

Solution 2 and 3: Use Target Networks

Q-learning with replay buffer and target network:

1. save target network parameters: $\phi' \leftarrow \phi$

$N\times$

2. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy, add it to $\mathcal{B}$

$K\times$

3. sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$ from $\mathcal{B}$

4. $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}'_i, \mathbf{a}'_i)])$

targets don't change in inner loop!

supervised regression

Slide credit S. Levine

“Classic” deep Q-learning algorithm (DQN)

Q-learning with replay buffer and target network:

1. save target network parameters: $\phi' \leftarrow \phi$
2. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy, add it to $\mathcal{B}$ — $N\times$
3. sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$ from $\mathcal{B}$ — $K\times$
4. $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}'_i, \mathbf{a}'_i)])$

“classic” deep Q-learning algorithm:

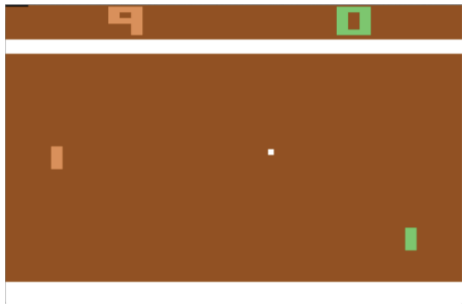
1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$, add it to $\mathcal{B}$
2. sample mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j, r_j\}$ from $\mathcal{B}$ uniformly
3. compute $y_j = r_j + \gamma \max_{\mathbf{a}'_j} Q_{\phi'}(\mathbf{s}'_j, \mathbf{a}'_j)$ using *target* network $Q_{\phi'}$
4. $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)(Q_\phi(\mathbf{s}_j, \mathbf{a}_j) - y_j)$

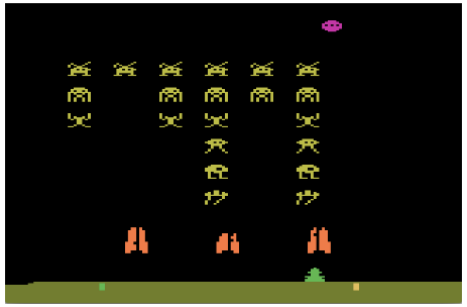
$K = 1$

5. update ϕ': copy ϕ every N steps

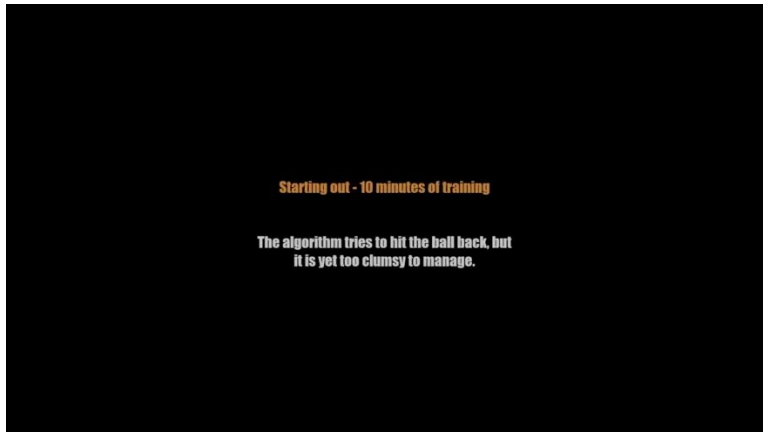
Slide credit S. Levine

DQN Surpasses Human Experts on (some) Atari Games

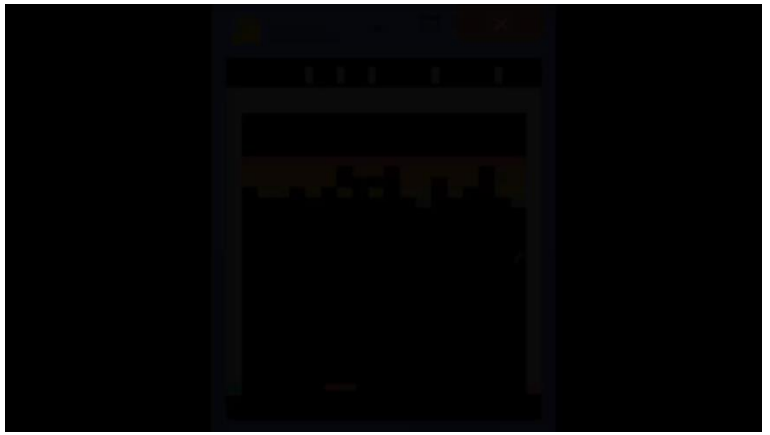




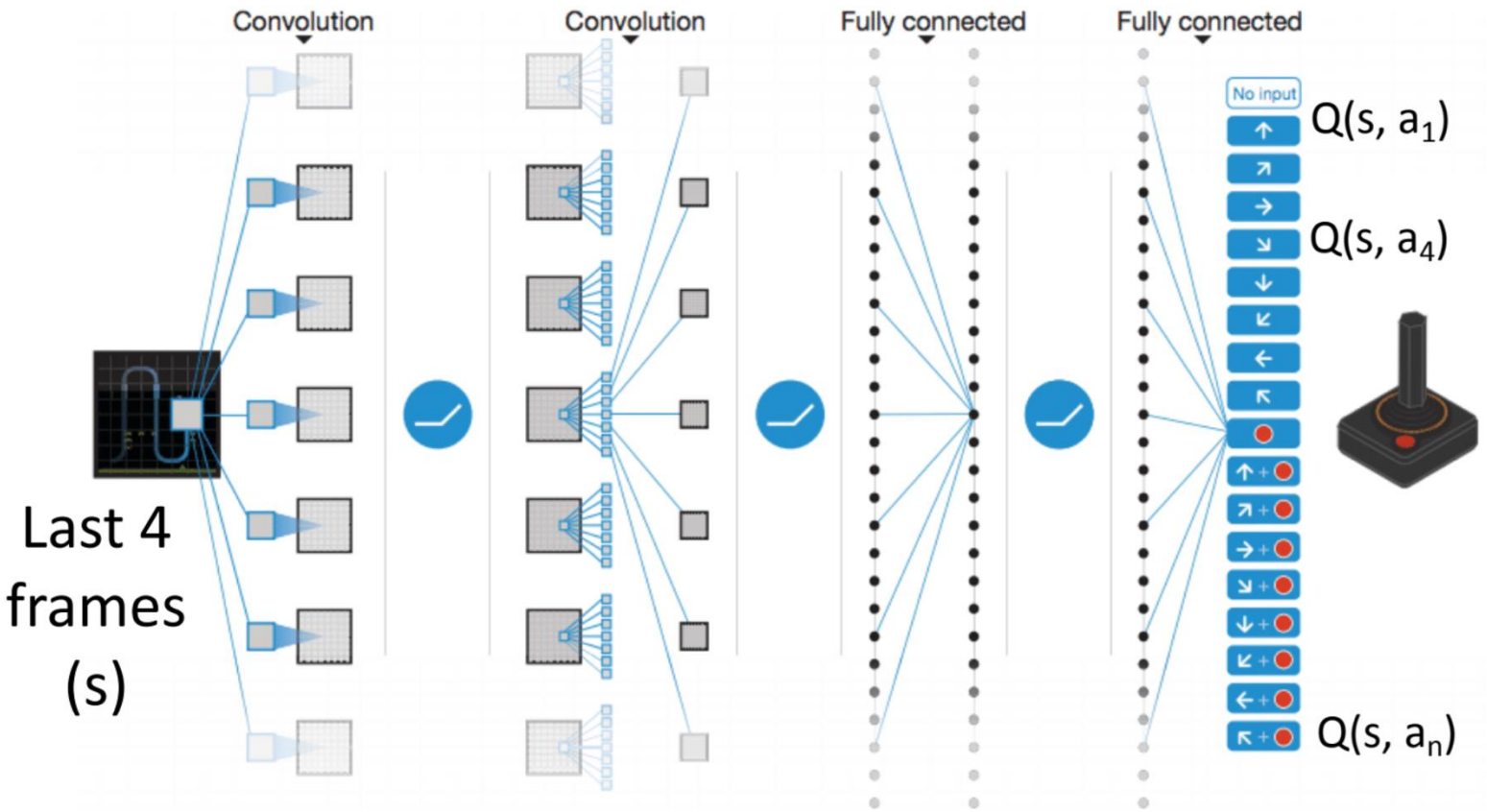






DQN Estimates State-Action Values

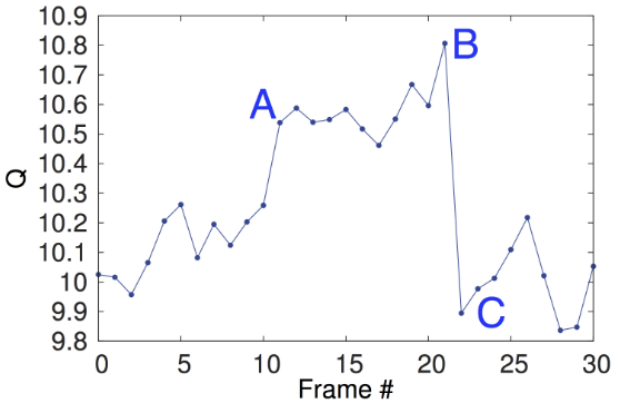


With probability $(1 - \varepsilon) \rightarrow$ execute $\max_a Q(s,a)$

With probability $\varepsilon \rightarrow$ execute random action

Image credit P. Agrawal

DQN Captures High-Value State-Action Pairs



A



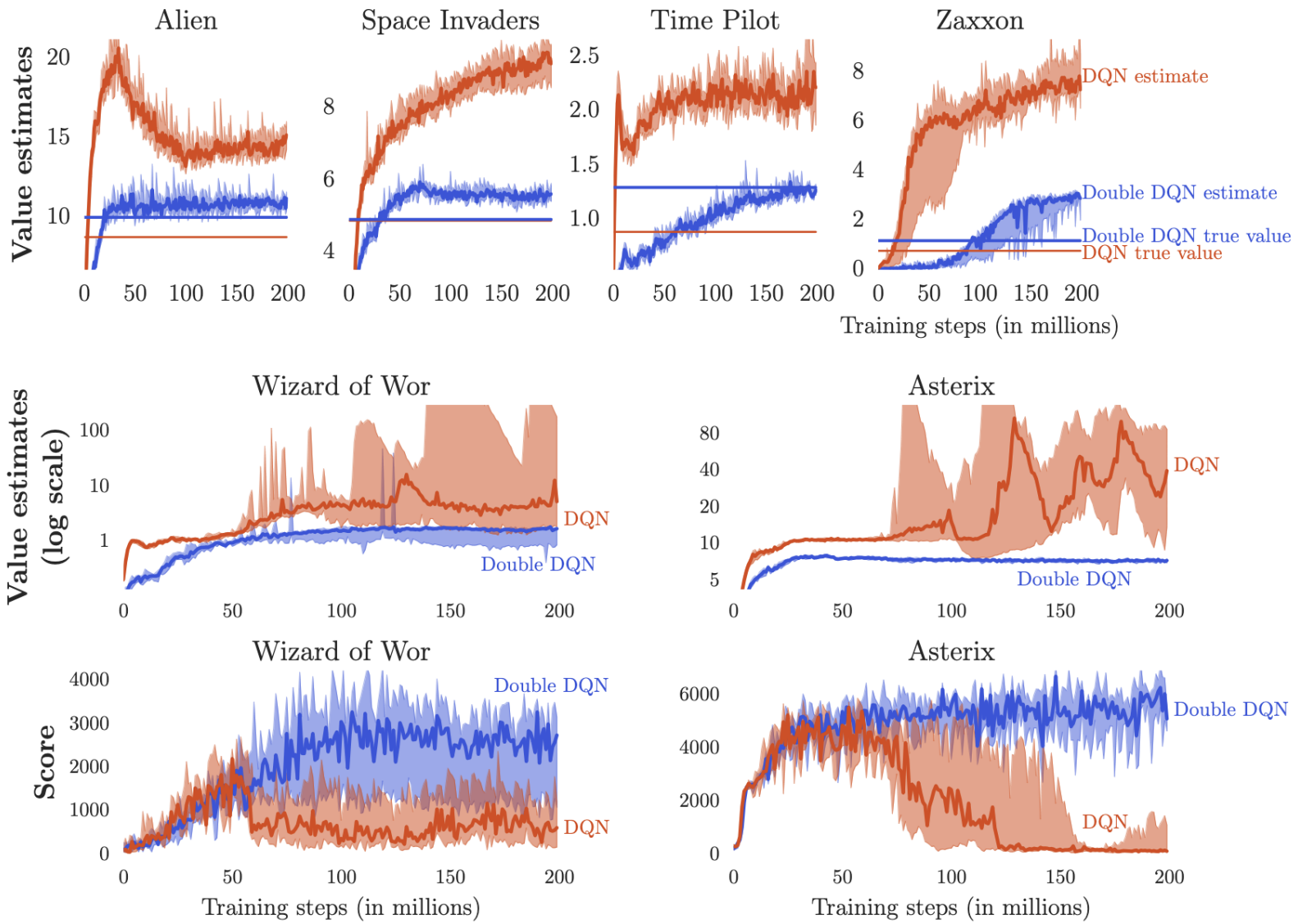
B



C



DQN Has a Problem in Overestimating Values



Check Appendix 1 for more details!

Content

- Introduction to Deep RL
- Deep Q-Learning
- Policy Gradient
  - ➢REINFORCE
  - ➢Actor-Critic Methods
  - ➢PPO

Wait! DQN Still Has a Problem…

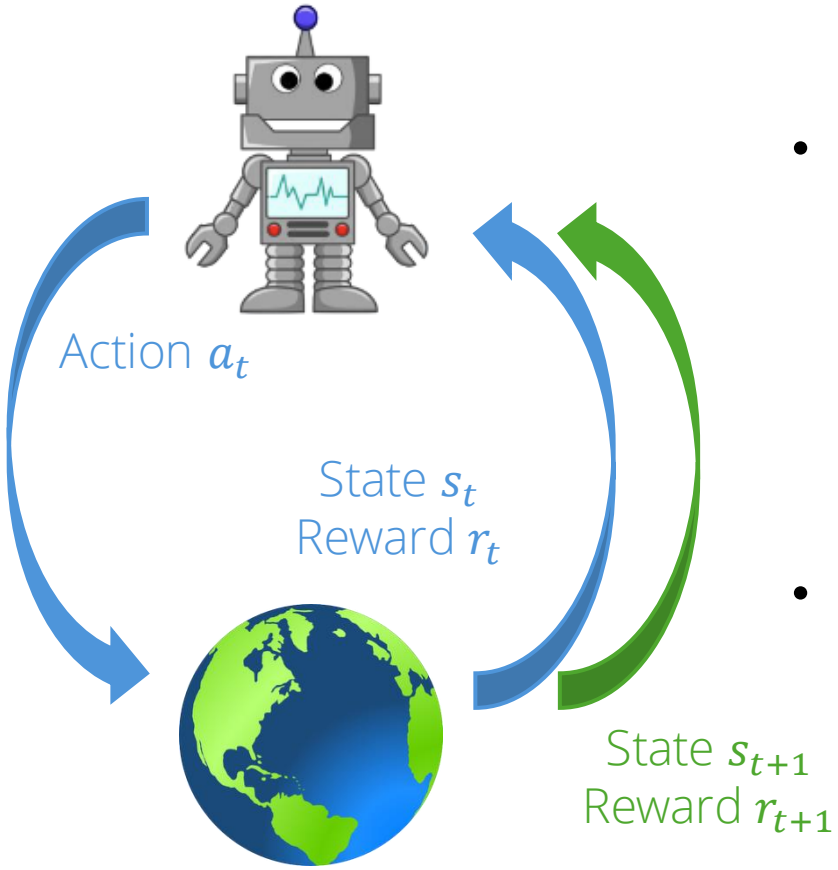
- Standard Q-learning:

$$y = r + \gamma \max_{a'} Q_{\phi'}(s', a') = r + \gamma Q_{\phi'}(s', \max_{a'} Q_{\phi'}(s', a'))$$

What if the action is continuous?

There're many solutions, but the most obvious one may be learning an action policy for maximizing the value…

Recap: Reinforcement Learning Aims to Maximize the Total Reward of an Episode of Interaction



- A trajectory of interaction in the environment

The policy

$$\underbrace{p_\theta(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T)}_{p_\theta(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$

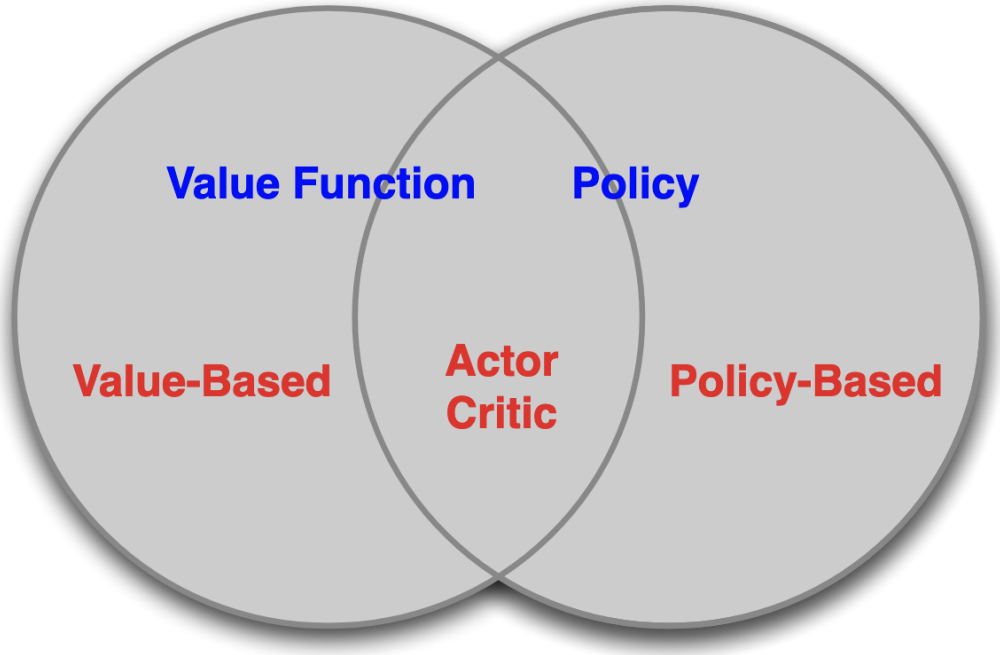
- Maximize the expected value of the cumulative sum of reward

$$\theta^\star = \arg\max_\theta E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Slide adapted from S. Levine

Valued-Based and Policy-Based RL

- Value Based
  - Learnt Value Function
  - Implicit policy (e.g. ϵ-greedy)
- Policy Based
  - No Value Function
  - Learnt Policy
- Actor-Critic
  - Learnt Value Function
  - Learnt Policy



Slide credit D. Silver

Policy Gradient

- Let $r(\tau) = \sum_t r(s_t, a_t)]$
- Let $J(\theta) = \sum_{\tau \sim p_\theta(\tau)} [\sum_t r(s_t, a_t)] = \int p_\theta(\tau) r(\tau) d\tau$
- Compute gradients of θ w.r.t to $J(\theta)$

$$\nabla J(\theta) = \int \nabla p_\theta(\tau) r(\tau) d\tau = \int p_\theta(\tau) \nabla \log p_\theta(\tau) r(\tau) d\tau = E_{\tau \sim p_\theta(\tau)} [\nabla \log p_\theta(\tau) r(\tau)]$$

$$\nabla \log p_\theta = \frac{1}{p_\theta} \nabla p_\theta$$

Policy Gradient

- Let $r(\tau) = \sum_t r(s_t, a_t)]$
- Let $J(\theta) = \sum_{\tau \sim p_\theta(\tau)}[\sum_t r(s_t, a_t)] = \int p_\theta(\tau) r(\tau) d\tau$
- Compute gradients of θ w.r.t to $J(\theta)$

$$\nabla J(\theta) = \int \nabla p_\theta(\tau) r(\tau) d\tau = \int p_\theta(\tau) \nabla \log p_\theta(\tau)\, r(\tau) d\tau = E_{\tau \sim p_\theta(\tau)}[\nabla \log p_\theta(\tau) r(\tau)]$$

$$\nabla \log p_\theta(\tau) = \nabla \log[p(s_0) \times \prod_{t=0}^{T} \pi_\theta(a_t | s_t) \times p(s_{t+1} | s_t, a_t)]$$

$$= \nabla[\log p(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t | s_t) + \sum_{t=0}^{T} \log(s_{t+1} | s_t, a_t)]$$

Policy Gradient

- Let $r(\tau) = \sum_t r(s_t, a_t)]$
- Let $J(\theta) = \sum_{\tau \sim p_\theta(\tau)}[\sum_t r(s_t, a_t)] = \int p_\theta(\tau) r(\tau) d\tau$
- Compute gradients of θ w.r.t to $J(\theta)$

$$\nabla J(\theta) = \int \nabla p_\theta(\tau) r(\tau) d\tau = \int p_\theta(\tau) \nabla \log p_\theta(\tau)\, r(\tau) d\tau = E_{\tau \sim p_\theta(\tau)}[\nabla \log p_\theta(\tau) r(\tau)]$$

$$\nabla \log p_\theta(\tau) = \nabla \log[p(s_0) \times \prod_{t=0}^{T} \pi_\theta(a_t | s_t) \times p(s_{t+1} | s_t, a_t)]$$

$$= \nabla[\log p(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t | s_t) + \sum_{t=0}^{T} \log(s_{t+1} | s_t, a_t)]$$

$$\nabla J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t | s_t) \right) \left(\sum_{t=0}^{T} r(s_t, a_t) \right) \right]$$

Policy Gradient

- Let $r(\tau) = \sum_t r(s_t, a_t)]$
- Let $J(\theta) = \sum_{\tau \sim p_\theta(\tau)}[\sum_t r(s_t, a_t)] = \int p_\theta(\tau) r(\tau) d\tau$
- Compute gradients of θ w.r.t to $J(\theta)$

$$\nabla J(\theta) = \int \nabla p_\theta(\tau) r(\tau) d\tau = \int p_\theta(\tau) \nabla \log p_\theta(\tau)\, r(\tau) d\tau = E_{\tau \sim p_\theta(\tau)}[\nabla \log p_\theta(\tau) r(\tau)]$$

$$\nabla \log p_\theta(\tau) = \nabla \log[p(s_0) \times \prod_{t=0}^{T} \pi_\theta(a_t | s_t) \times p(s_{t+1} | s_t, a_t)]$$

$$= \nabla[\log p(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t | s_t) + \sum_{t=0}^{T} \log(s_{t+1} | s_t, a_t)]$$

$$\nabla J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t | s_t) \right) \left(\sum_{t=0}^{T} r(s_t, a_t) \right) \right]$$

The transition function is not needed.
We just need experience!

Maximizing Action Trajectories using Returns

- Policy Gradients weights gradients with a sum of rewards of the trajectory:

$$\nabla J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t | s_t) \right) \left(\sum_{t=0}^{T} r(s_t, a_t) \right) \right]$$

The policy is optimized to follow the trajectory of high-reward episodes

- Policy Gradients learn from "trial and error"

Slide adapted from S. Levine

Maximizing Action Trajectories using Returns

- Policy Gradients weights gradients with a sum of rewards of the trajectory:

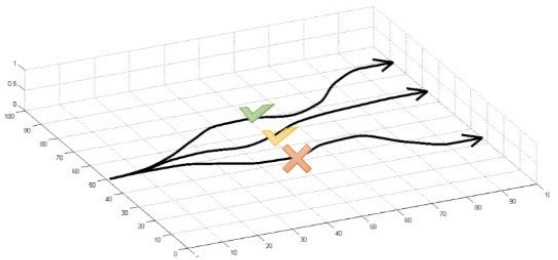
$$\nabla J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t | s_t) \right) \left(\sum_{t=0}^{T} r(s_t, a_t) \right) \right]$$

- Idea: approximate expected value by sampling

$$\nabla J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left[\left(\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t | s_t) \right) \left(\sum_{t=0}^{T} r(s_t, a_t) \right) \right]$$

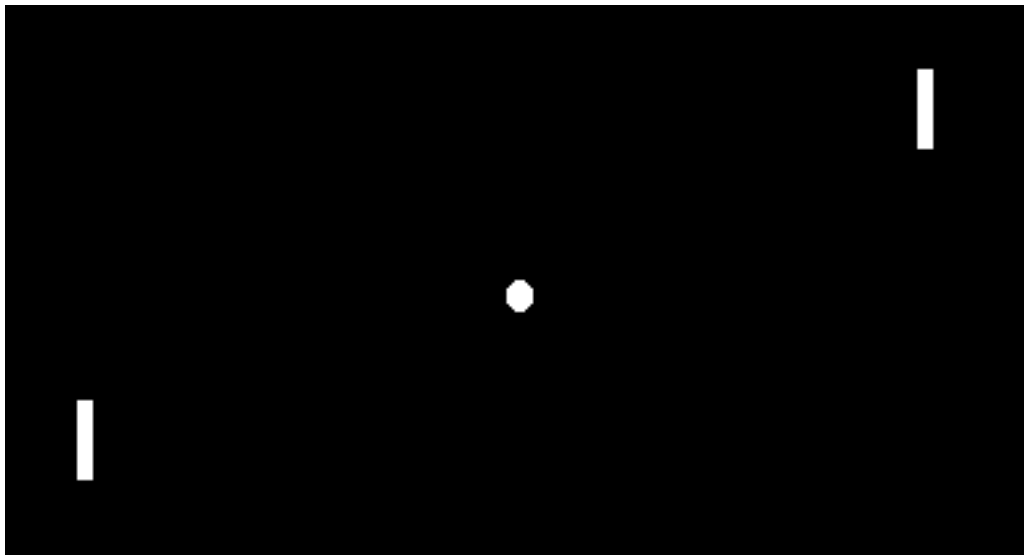
REINFORCE algorithm (Monte-Carlo Policy Gradient)

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

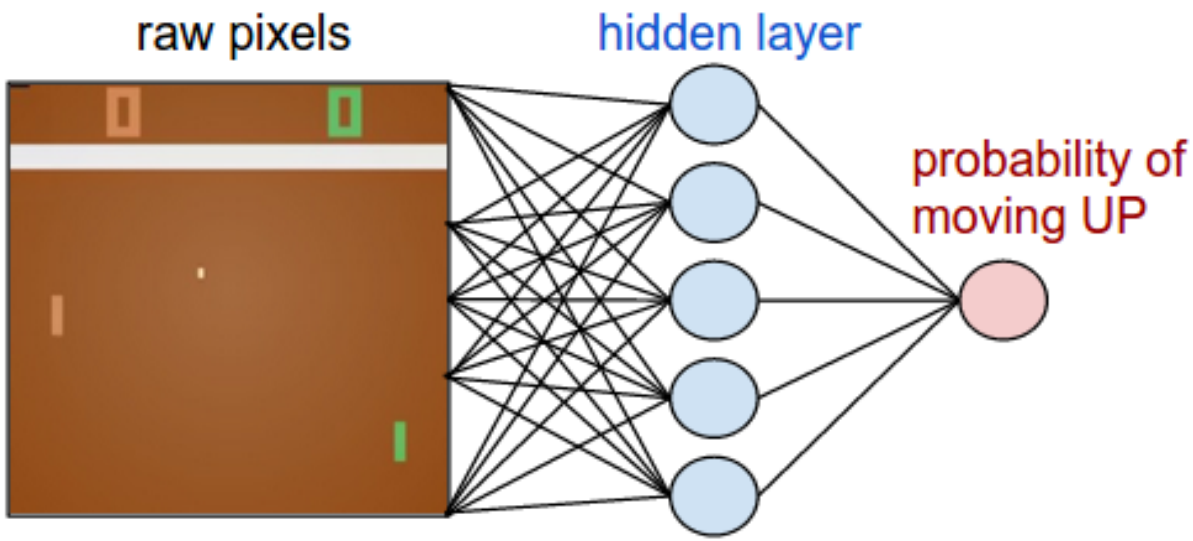


Slide adapted from S. Levine

An Example of Pong



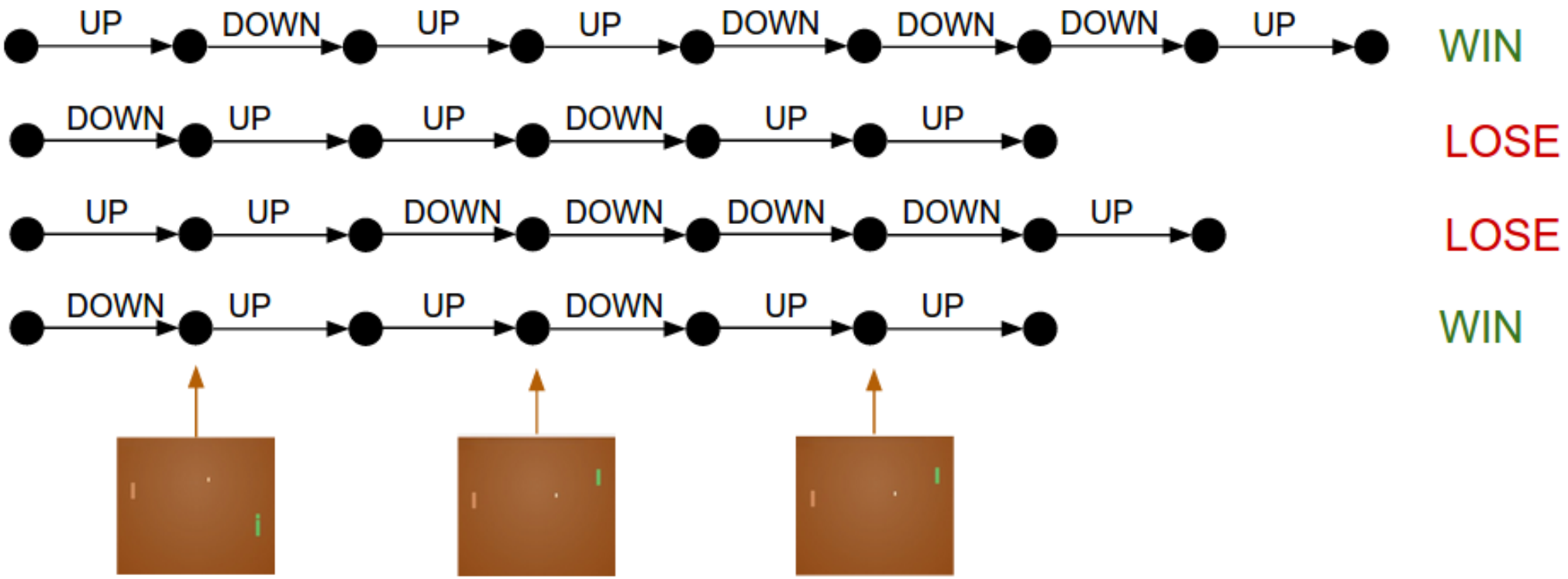
Policy network



Slide adapted from K. Fragkiadaki
Material Source: https://karpathy.github.io/2016/05/31/rl/

Reinforcement Learning: Optimize the Policy Based on Trial and Error

- Rollout the policy and collect episodes…



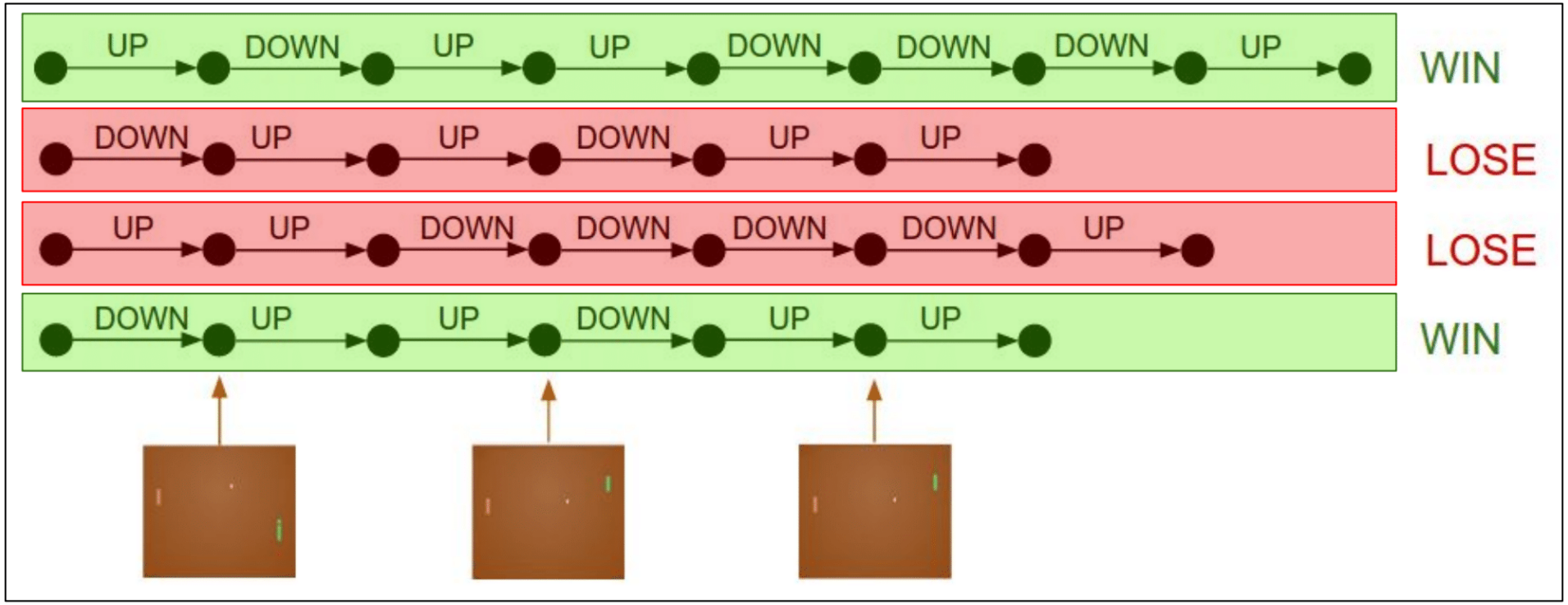
Material Source: https://karpathy.github.io/2016/05/31/rl/

Pretend every action we took here was the correct label.

maximize: $\log p(y_i \mid x_i)$

Pretend every action we took here was the wrong label.

maximize: $(-1) * \log p(y_i \mid x_i)$



We don't need any action labels!

$$\nabla_\theta U(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) R(\tau^{(i)})$$

Slide adapted from K. Fragkiadaki
Material Source: https://karpathy.github.io/2016/05/31/rl/

Wait! Policy Gradient Also Has Problems…

REINFORCE algorithm:



1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)\right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i)\right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Wait! Policy Gradient Also Has Problems…

REINFORCE algorithm:

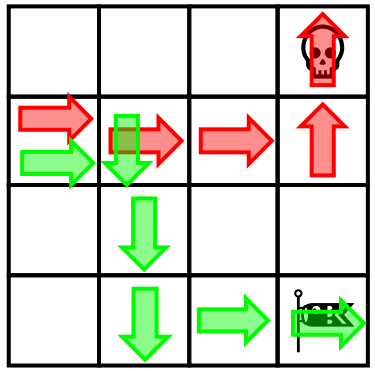
1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)\right)\left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i)\right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Gradients of each action in the trajectory are weighted by the sum of rewards

Wait! Policy Gradient Also Has Problems…

REINFORCE algorithm:

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)\right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i)\right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

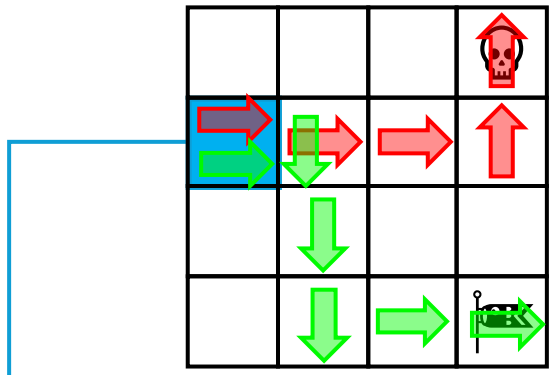


- The gradient estimator is unbiased, but requires a very large number of samples to accurately approximate the true gradient
- When the number of sample is small, the variance is high…

Gradients Have High Variance

REINFORCE algorithm:

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)\right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i)\right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

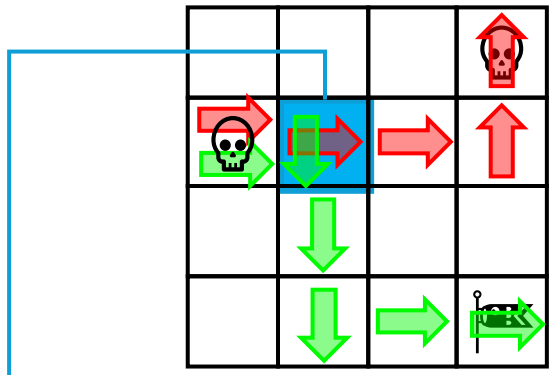


Gradients of the same action has high variance!

Gradients Have High Variance

REINFORCE algorithm:

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)\right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i)\right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

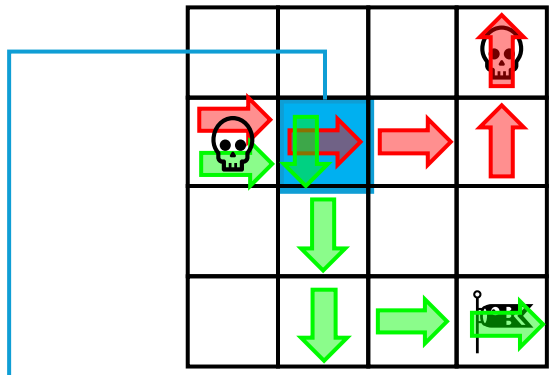


Gradients of current actions forced to account for previous success/failure

Gradients Have High Variance

REINFORCE algorithm:

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)\right)\left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i)\right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



Gradients of current actions forced to account for previous success/failure

Solution:

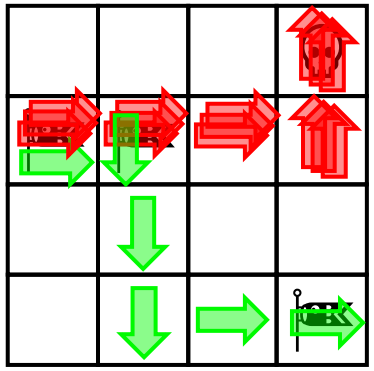
- Causality: let current actions only account for future success/failure

$$\nabla J(\theta) = E_{\tau \sim p_\theta(\tau)}\left[\left(\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t|s_t)\right)\left(\sum_{t'=t}^{T} \gamma^{t'} r(s_{t'}, a_{t'})\right)\right]$$

Gradients Have High Variance

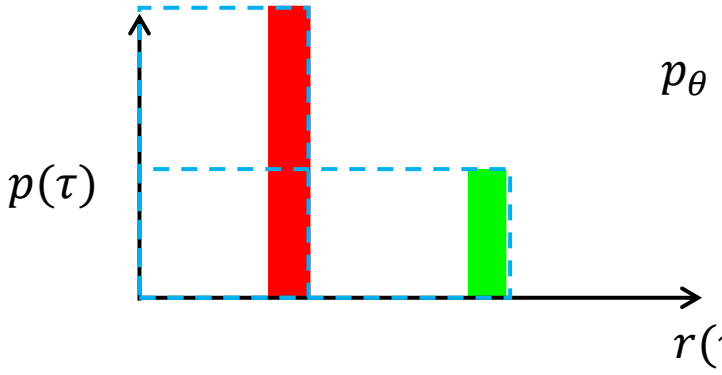
REINFORCE algorithm:

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)\right)\left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i)\right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



$$\nabla J(\theta) = \int p_\theta(\tau) \nabla \log p_\theta(\tau)\, r(\tau) d\tau$$

$$p_\theta(\tau^r) r(\tau^r) = p_\theta(\tau^g) r(\tau^g)$$



Common sub-optimal trajectory receives same weightings as rare optimal trajectory

A More General Solution to Reduce Variance

- Subtract a baseline b

$$\nabla J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N} \nabla \log p_\theta(\tau)\,[r(\tau) - b]$$

- The gradients don't change

$$E[\nabla \log p_\theta(\tau) b] = \int p_\theta(\tau) \nabla \log p_\theta(\tau)\, b d\tau = \int \nabla p_\theta(\tau) b d\tau = b \nabla \int p_\theta(\tau) d\tau = 0$$

- Subtracting a baseline is unbiased in expectation!

What about variance?

$$\mathrm{Var}[x] = E[x^2] - E[x]^2$$

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\theta(\tau)}[\nabla_\theta \log p_\theta(\tau)(r(\tau) - b)]$$

$$\mathrm{Var} = E_{\tau \sim p_\theta(\tau)}[(\nabla_\theta \log p_\theta(\tau)(r(\tau) - b))^2] - E_{\tau \sim p_\theta(\tau)}\underbrace{[\nabla_\theta \log p_\theta(\tau)(r(\tau) - b)]}^2$$

this bit is just $E_{\tau \sim p_\theta(\tau)}[\nabla_\theta \log p_\theta(\tau) r(\tau)]$

(baselines are unbiased in expectation)

Slide credit S. Levine

What about variance?

$$\mathrm{Var}[x] = E[x^2] - E[x]^2$$

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\theta(\tau)}[\nabla_\theta \log p_\theta(\tau)(r(\tau) - b)]$$

$$\mathrm{Var} = E_{\tau \sim p_\theta(\tau)}[(\nabla_\theta \log p_\theta(\tau)(r(\tau) - b))^2] - E_{\tau \sim p_\theta(\tau)}[\underbrace{\nabla_\theta \log p_\theta(\tau)(r(\tau) - b)}]^2$$

this bit is just $E_{\tau \sim p_\theta(\tau)}[\nabla_\theta \log p_\theta(\tau) r(\tau)]$
(baselines are unbiased in expectation)

$$\frac{d\mathrm{Var}}{db} = \frac{d}{db} E[g(\tau)^2 (r(\tau) - b)^2] = \frac{d}{db} \left(E[g(\tau)^2 r(\tau)^2] - 2E[g(\tau)^2 r(\tau) b] + b^2 E[g(\tau)^2] \right)$$

$$= -2E[g(\tau)^2 r(\tau)] + 2bE[g(\tau)^2] = 0$$

$$b = \frac{E[g(\tau)^2 r(\tau)]}{E[g(\tau)^2]}$$

This is just expected reward, but weighted by gradient magnitudes!

Slide credit S. Levine

Other Options of Baselines?

- Constant baseline $b = \frac{1}{N}\sum_{i=1}^{N} r(\tau)$

$$\nabla J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N} \nabla \log p_\theta(\tau)\,[r(\tau) - b]$$

- Time-dependent baseline $b_t = \frac{1}{N}\sum_{i=1}^{N} r_t(\tau)$

$$\nabla J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T} \nabla \log p_\theta(a_t|s_t)\,[r_t(\tau_t) - b_t]$$

- State-dependent baseline $b_s = V^\pi(s)$

$$\nabla J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T} \nabla \log p_\theta(a_t|s_t)\,\left[r_t(\tau_t) - b_{s_t}\right]$$

Slide adapted from K. Fragkiadaki

Content

- Introduction to Deep RL
- Deep Q-Learning
- Policy Gradient
  - REINFORCE
  - Actor-Critic Methods
  - PPO

Actor Critic

- Policy Gradient:

$$\nabla J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t | s_t) \right) \left(\sum_{t=0}^{T} r(s_t, a_t) - b \right) \right]$$

- Re-write with action value $Q_\pi(s, a)$:

$$\nabla J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t | s_t) \right) (Q^\pi(s_t, a_t) - b) \right]$$

Actor Critic

- Re-write with action value $Q_\pi(s,a)$:

$$\nabla J(\theta) = E_{\tau\sim p_\theta(\tau)}\left[\left(\sum_{t=0}^{T}\nabla\log\pi_\theta(a_t\,|s_t)\right)(Q^\pi\,(s_t,a_t)-b)\right]$$

$$\text{Let } b = V^\pi(s) \Rightarrow \nabla J(\theta) = E_{\tau\sim p_\theta(\tau)}\left[\left(\sum_{t=0}^{T}\nabla\log\pi_\theta(a_t\,|s_t)\right)(\underbrace{Q^\pi(s_t,a_t)-V^\pi(s)}_{\text{Advantage } A^\pi(s_t,a_t)})\right]$$

- Advantage $A^\pi\,(s_t,a_t)$ measures how much better action a_t is than other actions

- If we learn a value estimator $V_\phi^\pi(s)$ parameterized by ϕ, we obtain:
  - Estimated action value $Q^\pi(s_t,a_t) = R(s_t,a_t)+\gamma V_\phi^\pi(s_{t+1})$
  - Estimated advantage $A^\pi(s_t,a_t) = R(s_t,a_t)+\gamma V_\phi^\pi(s_{t+1}) - V_\phi^\pi(s_t)$

On-policy Advantage Actor Critic

0. Initialize policy parameters θ and critic parameters ϕ .

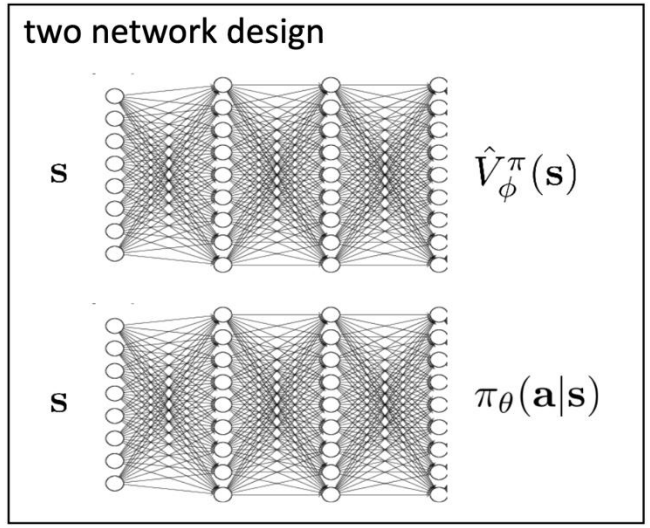
1. Sample trajectories $\{\tau_i = \{s_t^i, a_t^i\}_{t=0}^T\}$ by deploying the current policy $\pi_\theta(a_t \mid s_t)$.

2. Fit value function $V_\phi^\pi(s)$ by MC or TD estimation (update ϕ)

3. Compute action advantages $A^\pi(s_t^i, a_t^i) = R(s_t^i, a_t^i) + \gamma V_\phi^\pi(s_{t+1}^i) - V_\phi^\pi(s_t^i)$

4. $\nabla_\theta J(\theta) \approx \hat{g} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) A^\pi(s_t^i, a_t^i)$

5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



Slide credit K. Fragkiadaki

On-policy Advantage Actor Critic

0. Initialize policy parameters θ and critic parameters ϕ .

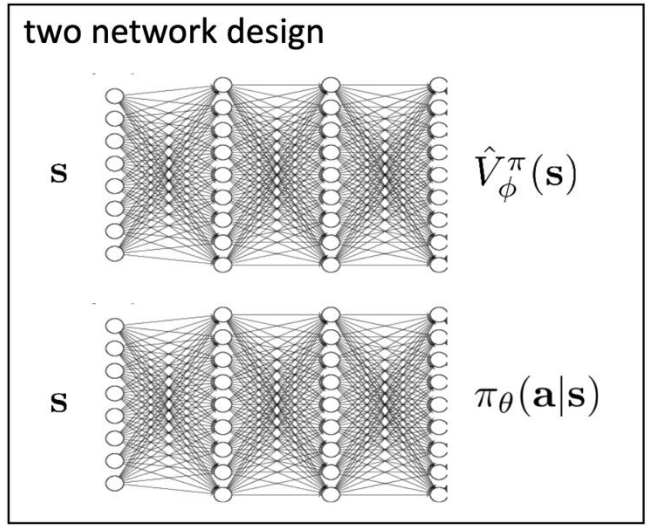
1. Sample trajectories $\{\tau_i = \{s_t^i, a_t^i\}_{t=0}^T\}$ by deploying the current policy $\pi_\theta(a_t \mid s_t)$.

2. Fit value function $V_\phi^\pi(s)$ by MC or TD estimation (update ϕ)

3. Compute action advantages $A^\pi(s_t^i, a_t^i) = R(s_t^i, a_t^i) + \gamma V_\phi^\pi(s_{t+1}^i) - V_\phi^\pi(s_t^i)$

4. $\nabla_\theta J(\theta) \approx \hat{g} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) A^\pi(s_t^i, a_t^i)$

5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



Slide credit K. Fragkiadaki

Off-policy Actor Critic with Importance Sampling

- Policy gradient objective:

$$J(\theta) = \sum_{\tau \sim p_\theta(\tau)} \sum_t r(s_t, a_t)$$

- Actor-critic objective:

$$J(\theta) = \sum_{\tau \sim p_\theta(\tau)} A^\pi(s_t, a_t)$$

- Off-policy Actor-critic objective with importance sampling:

$$J(\theta) = \sum_{\tau \sim p_{\theta_{old}}(\tau)} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A^\pi(s_t, a_t)$$

Re-scale the value estimates of old policy for the current policy

Check Appendix 7 in Lecture 6!

Content

- Introduction to Deep RL
- Deep Q-Learning
- Policy Gradient
  - REINFORCE
  - Actor-Critic Methods
  - PPO

Wait! Actor Critic Methods Have Issues

- Challenge 1: policy gradient calculates the steepest ascent direction for the rewards. First-order approximation assumes flat surface, leading to horrible updates if the surface has high curvature

$$\underbrace{\nabla J(\theta)}_{\text{Steepest direction to maximize rewards}} = E_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t | s_t) \right) \mathrm{A}^{\pi_\theta}(s_t, a_t) \right]$$

$$\theta_{k+1} = \theta_k + \alpha \nabla J(\theta)$$





Slide adapted from J. Hui

Image source

Wait! Actor Critic Methods Have Issues

- Challenge 1: policy gradient calculates the steepest ascent direction for the rewards. First-order approximation assumes flat surface, leading to horrible updates if the surface has high curvature

$$\underbrace{\nabla J(\theta)}_{\text{Steepest direction to maximize rewards}} = E_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t | s_t) \right) \mathrm{A}^{\pi_\theta}(s_t, a_t) \right]$$

$$\theta_{\mathrm{k+1}} = \theta_{\mathrm{k}} + \alpha \nabla J(\theta)$$

- Challenge 2: tuning learning rate is tricky. Low learning rate makes training slow; high learning rate breaks the policy.

Slide adapted from J. Hui

Idea: Trust Region Policy Optimization (TRPO)

Idea: TRPO iteratively updates the policy by solving the following:

In each iteration k, the policy is updated from π_{θ_k} to $\pi_{\theta_{k+1}}$

$$\pi_{\theta_{k+1}} = \arg\max_{\theta} \mathbb{E}_{s\sim d_{\mu}^{\pi_{\theta_k}}, a\sim\pi_{\theta_k}(\cdot|s)}\left[\frac{\pi_{\theta}(a\,|\,s)}{\pi_{\theta_k}(a\,|\,s)} A^{\pi_{\theta_k}}(s,a;)\right]$$

subject to $D(\pi_{\theta_k}, \pi_{\theta}) \leq \delta$ ← "trust region"



TRPO wants to achieve policy improvement via constrained optimization

Slide credit Ping-Chun Hsieh

Trust Region

- Line search: determine the ascending / descending direction and take a step towards that direction (e.g. gradient descent)
- Trust region: determine the maximum step size that we want to explore and locate the optimal point within this trust region. We can adapt the step size (area of trust region) based on the goodness of approximative function $\mathbf{M_i}$

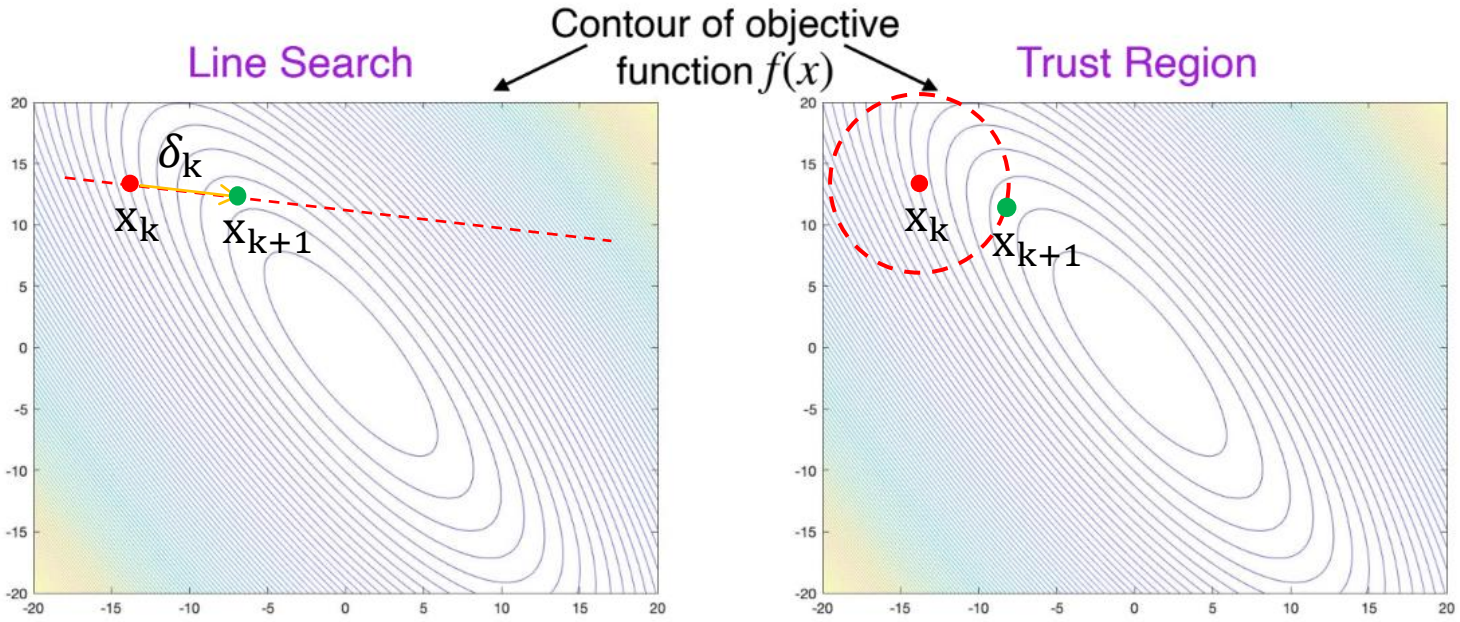


Image source Ping-Chun Hsieh

Trust Region

$$\max_{s} m_k(s)$$
$$s.t. \|s\| \leq \delta$$

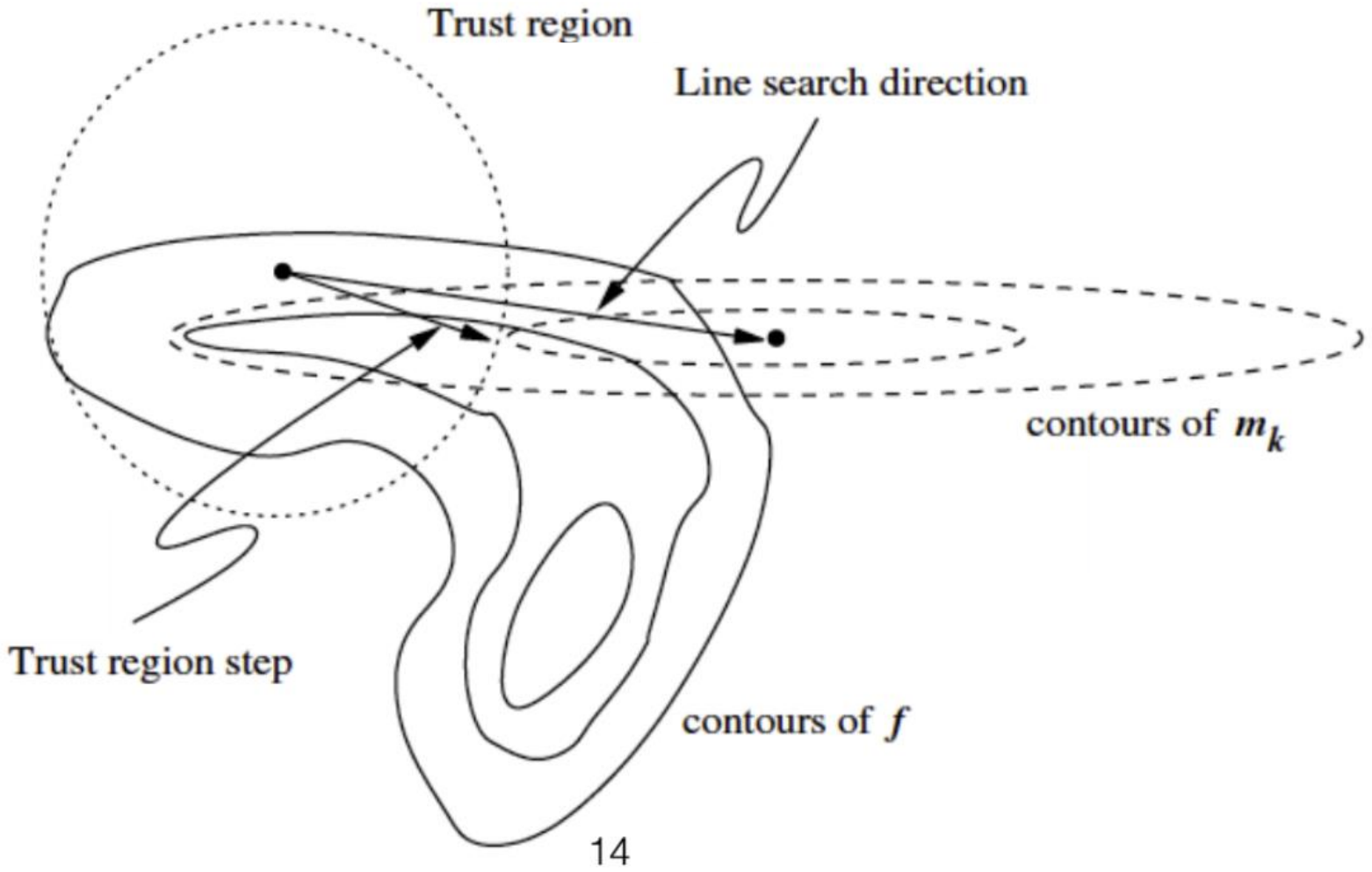


Image source Ping-Chun Hsieh

Off-policy Actor Critic with Importance Sampling

- Off-policy Actor-critic objective with importance sampling:

$$J(\theta) = \sum_{\tau \sim p_{\theta_{old}}(\tau)} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A^{\pi_{\theta_{old}}}(s_t, a_t)$$

- Policy gradients are:

Refine the current policy

$$\nabla J(\theta) = E_{\tau \sim p_{\pi_{\theta_{old}}}(\tau)} \left[\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t|s_t) \left(\prod_{t'=0}^{t} \frac{\pi_\theta(a_{t'}|s_{t'})}{\pi_{\theta_{old}}(a_{t'}|s_{t'})} \right) A^{\pi_{\theta_{old}}}(s_t, a_t) \right]$$

Sample data from other policies

Importance Sampling Results in High Variance

Importance sampling is a technique for estimating expectations using samples drawn from a different distribution.

$$\mathop{\mathrm{E}}_{x\sim P}[f(x)] = \mathop{\mathrm{E}}_{x\sim Q}\left[\frac{P(x)}{Q(x)}f(x)\right] \approx \frac{1}{|D|}\sum_{x\in D}\frac{P(x)}{Q(x)}f(x), \quad D\sim Q$$

The ratio $P(x)/Q(x)$ is the **importance sampling weight** for x.

What is the variance of an importance sampling estimator?

The variance may explode if the current policy is very different from the behavior policy

$$\begin{aligned}
\mathrm{var}(\hat{\mu}_Q) &= \frac{1}{N}\mathrm{var}\left(\frac{P(x)}{Q(x)}f(x)\right) \\
&= \frac{1}{N}\left(\mathop{\mathrm{E}}_{x\sim Q}\left[\left(\frac{P(x)}{Q(x)}f(x)\right)^2\right] - \mathop{\mathrm{E}}_{x\sim Q}\left[\frac{P(x)}{Q(x)}f(x)\right]^2\right) \\
&= \frac{1}{N}\left(\mathop{\mathrm{E}}_{x\sim P}\left[\frac{P(x)}{Q(x)}f(x)^2\right] - \mathop{\mathrm{E}}_{x\sim P}[f(x)]^2\right)
\end{aligned}$$

The term in red is problematic—if $P(x)/Q(x)$ is large in the wrong places, the variance of the estimator explodes.

Slide credit Joshua Achiam

Objective Function using Importance Sampling

- Objective without importance sampling: $J(\theta) = \sum_{\tau \sim p_\theta(\tau)} A^{\pi_\theta}(s_t, a_t)$
- Objective with importance sampling: $\mathrm{J}^{\mathrm{IS}}(\theta) = \sum_{\tau \sim p_{\theta_{old}}(\tau)} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A^{\pi_{\theta_{old}}}(s_t, a_t)$
- Same derivatives if the current policy is close to the behavior policy

$$\nabla_\theta J^{IS}(\theta)\Big|_{\theta_{old}} = \sum_{\tau \sim p_{\theta_{old}}(\tau)} \frac{\nabla_\theta \pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\Big|_{\theta_{old}} A^{\pi_{\theta_{old}}}(s_t, a_t) = \sum_{\tau \sim p_{\theta_{old}}(\tau)} \frac{\pi_\theta \nabla_\theta \log \pi_\theta}{\pi_{\theta_{old}}}\Big|_{\theta_{old}} A^{\pi_{\theta_{old}}}(s_t, a_t)$$

$$\approx \sum_{\tau \sim p_{\theta_{old}}(\tau)} \nabla_\theta \log \pi_\theta \Big|_{\theta_{old}} A^{\pi_{\theta_{old}}}(s_t, a_t) = \nabla_\theta J(\theta) \Big|_{\theta_{old}}$$

Slide adapted from J. Hui

Learning Objective

- Our goal: Optimize to achieve consistent policy improvement

  Reformulate the optimization problem:

$$\max_{\pi} \mathrm{J}(\pi) - \mathrm{J}(\pi_{\mathrm{old}})$$

Learning Objective

- Our goal: Optimize to achieve consistent policy improvement

  Reformulate the optimization problem:

$$\max_{\pi} \mathrm{J}(\pi) - \mathrm{J}(\pi_{\mathrm{old}})$$

  Solve this difficult optimization problem with the MM algorithm! What is the approximative function $\mathbf{M}$?

- Our observation: Off-policy actor critic allows sample data to be used efficiently, however, the current policy should not be very different from the behavior policy

Slide adapted from J. Hui

The Lower Bound of the Learning Objective

- Let's first define a function $\mathcal{L}$:

$$\mathcal{L}_{\pi_{old}}(\pi) = \frac{1}{1-\gamma} \underset{s\sim d^{\pi_{old}}, a\sim\pi_{old}}{E}\left[\frac{\pi(a|s)}{\pi_{old}(a|s)} A^{\pi_{old}}(\mathrm{s},\mathrm{a})\right] = \underset{\tau\sim\pi_{old}}{E}\left[\sum_{t=0}^{\infty} \gamma^t \frac{\pi(a|s)}{\pi_{old}(a|s)} A^{\pi_{old}}(\mathrm{s},\mathrm{a})\right]$$

where $d^{\pi_{old}}$ is the state visit frequency under policy π_{old}

$$d^{\pi_{old}} = (1-\gamma)\sum_{t=0}^{\infty} \gamma^t P(s_t = s|\pi_{old})$$

Slide adapted from J. Hui

The Lower Bound of the Learning Objective

- Let's first define a function $\mathcal{L}$:

$$\mathcal{L}_{\pi_{old}}(\pi) = \frac{1}{1-\gamma} \underset{s\sim d^{\pi_{old}}, a\sim\pi_{old}}{E}\left[\frac{\pi(a|s)}{\pi_{old}(a|s)} A^{\pi_{old}}(\mathrm{s},\mathrm{a})\right] = \underset{\tau\sim\pi_{old}}{E}\left[\sum_{t=0}^{\infty} \gamma^t \frac{\pi(a|s)}{\pi_{old}(a|s)} A^{\pi_{old}}(\mathrm{s},\mathrm{a})\right]$$

where $d^{\pi_{old}}$ is the discounted state visit frequency under policy π_{old}

$$d^{\pi_{old}} = (1-\gamma)\sum_{t=0}^{\infty} \gamma^t P(s_t = s|\pi_{old})$$

- One can show that (check TRPO paper)

$$\left|\mathrm{J}(\pi) - (\mathrm{J}(\pi_{\mathrm{old}}) + \mathcal{L}_{\pi_{old}}(\pi))\right| \leq \mathrm{C}\sqrt{\underset{s\sim d^{\pi_{old}}}{E}[D_{KL}(\pi|\pi_{old})[\mathrm{s}]]}$$

$$\Rightarrow \mathrm{J}(\pi) - \mathrm{J}(\pi_{\mathrm{old}}) \geq \mathcal{L}_{\pi_{old}}(\pi) - \mathrm{C}\sqrt{\underset{s\sim d^{\pi_{old}}}{E}[D_{KL}(\pi|\pi_{old})[\mathrm{s}]]}$$

→ Lower bound function

Slide adapted from J. Hui

The Lower Bound of the Learning Objective

- The lower bound of the learning objective:

$$\mathrm{J}(\pi) - \mathrm{J}(\pi_{\mathrm{old}}) \geq \mathcal{L}_{\pi_{old}}(\pi) - \mathrm{C}\sqrt{\underset{s\sim d^{\pi_{old}}}{E}[D_{KL}(\pi|\pi_{old})[s]]}$$

Guarantee improvement if the policy get slightly better every update

- Multiple algorithms to solve the lower bound:
  - Natural Policy Gradient
  - Trust Region Policy Optimization
  - Proximal Policy Optimization

Check the Appendix 2 and their original papers!

Slide credit Joshua Achiam

Proximal Policy Optimization (PPO)

- The lower bound of the learning objective:

$$\mathrm{J}(\pi) - \mathrm{J}(\pi_{\mathrm{old}}) \geq \mathcal{L}_{\pi_{old}}(\pi) - \mathrm{C}\sqrt{\underset{\mathrm{s}\sim d^{\pi_{old}}}{E}[D_{KL}(\pi|\pi_{old})[\mathrm{s}]]}$$

Lower bound function

- Maximization of the lower bound function

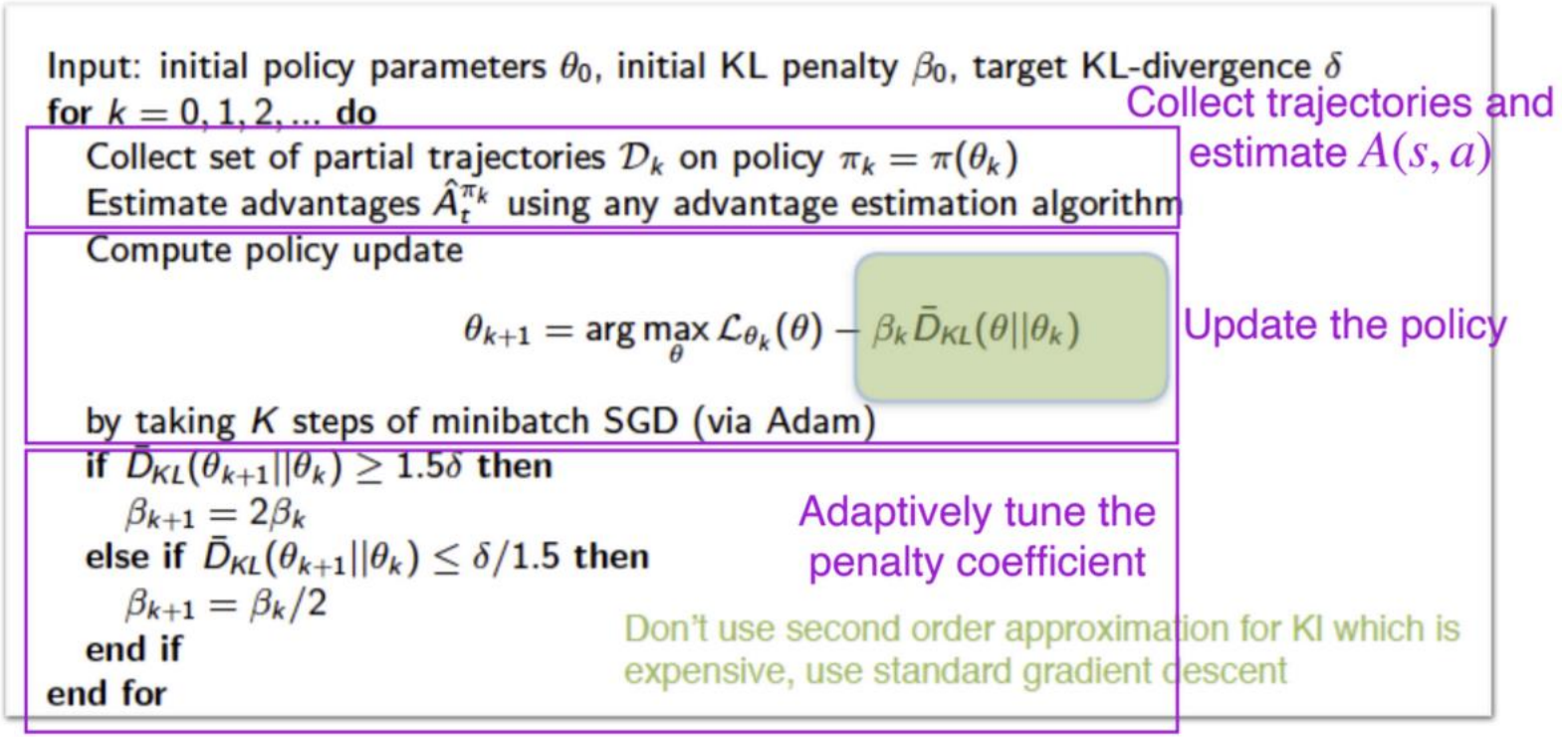
KL penalized formulation

$$\max_{\pi} \mathcal{L}_{\pi_{old}}(\pi) - \mathrm{C}\sqrt{\underset{\mathrm{s}\sim d^{\pi_{old}}}{E}[D_{KL}(\pi|\pi_{old})[\mathrm{s}]]}$$

Slide credit Joshua Achiam

Adaptive KL Penalty

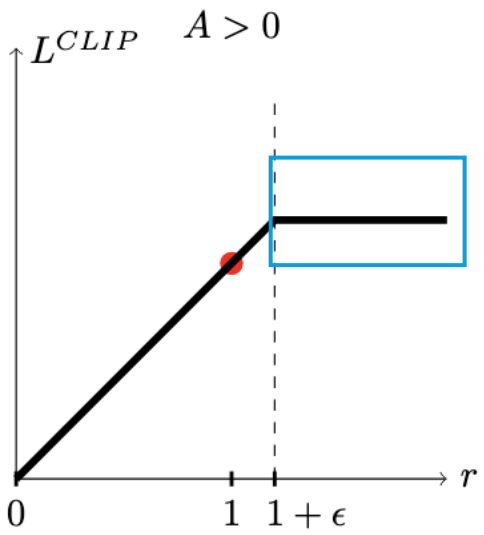
- KL penalized formulation: $\theta_{k+1} = \arg\max_{\theta} \mathcal{L}_{\theta_k}(\theta) - \bar{D}_{KL}(\theta|\theta_k)$
- Penalty coefficient β_k changes between iterations to approximately enforce KL-divergence constraint

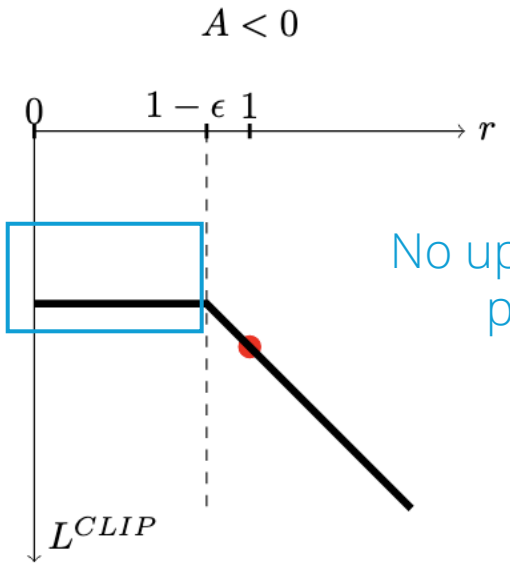


Slide credit Joshua Achiam and Ping-Chun Hsieh

Clipped Objective

- Let $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}$ be the ratio of the current policy π_θ and old policy π_{θ_k}
- New objective: $\mathcal{L}^{CLIP}_{\theta_k}(\theta) = \underset{\tau\sim\pi_k}{E}\left[\min\left(r_t(\theta)\widehat{A}^{\pi_k}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}^{\pi_k}_t\right)\right]$
- Policy update: $\theta_{k+1} = \arg\max_\theta \mathcal{L}^{CLIP}_{\theta_k}(\theta)$





No update if the current and old policy are very different

Slide credit Joshua Achiam

Clipped Objective

Input: initial policy parameters θ_0, clipping threshold ϵ

Collect trajectories and estimate $A(s, a)$

for $k = 0, 1, 2, \ldots$ **do**

Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Compute policy update

$$\theta_{k+1} = \arg\max_{\theta} \mathcal{L}_{\theta_k}^{CLIP}(\theta)$$

by taking K steps of minibatch SGD (via Adam), where

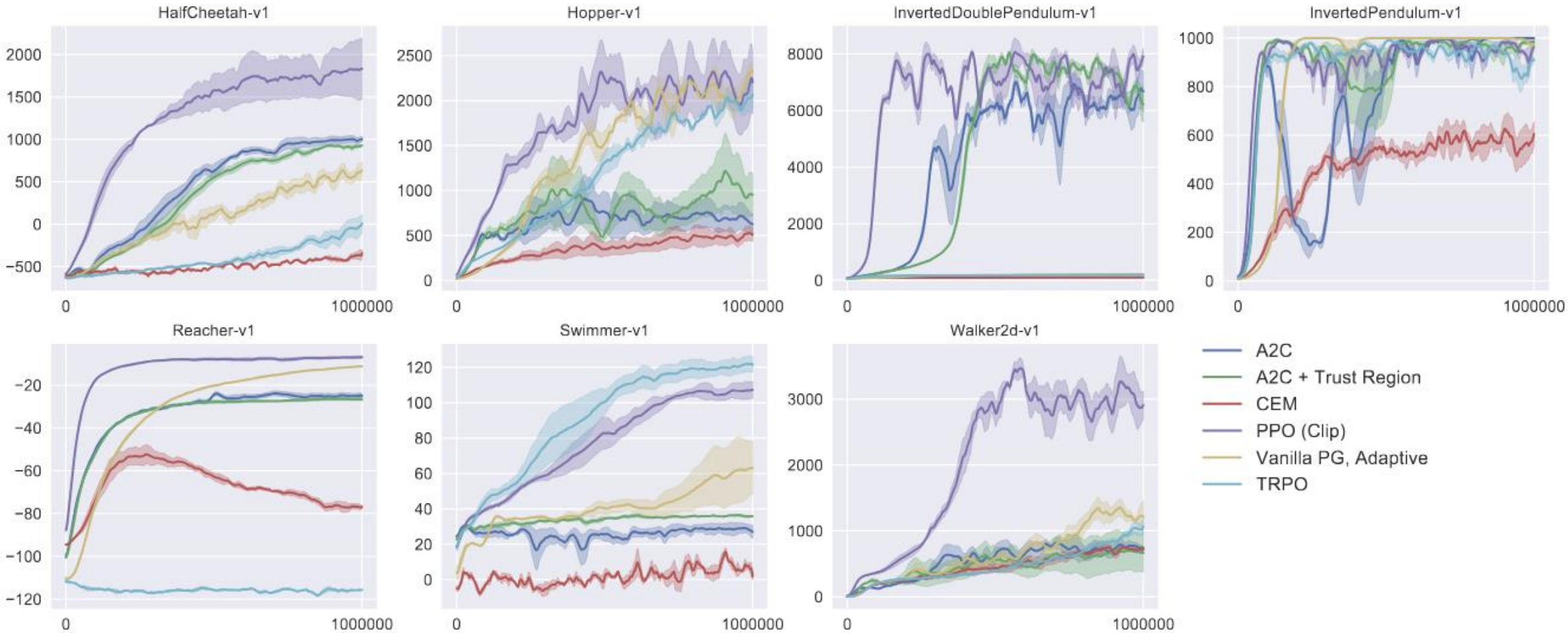
$$\mathcal{L}_{\theta_k}^{CLIP}(\theta) = \mathop{\mathrm{E}}_{\tau \sim \pi_k} \left[\sum_{t=0}^{T} \left[\min(r_t(\theta)\hat{A}_t^{\pi_k}, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t^{\pi_k}) \right] \right]$$

end for

PPO does not use 2nd-order derivatives

Slide credit Ping-Chun Hsieh

PPO is a Strong Baseline for Many RL Tasks



Reinforcement Learning vs. Imitation Learning

- Imitation learning:
  - Pros: IL is sample efficient
    1. It bootstraps from human knowledge
    2. It produces an ok-ish model from a **relatively** small dataset
  - Cons: IL fails to generalize
    1. It struggles with distributional drift

- Reinforcement learning:
  - Pros: RL is able to generalize
    1. It automatically discovers feasible strategies by trial and error
  - Cons: RL is sample inefficient
    1. It requires a lot of sample data before discovering the solution